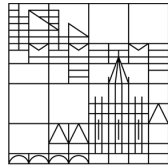


Universität
Konstanz



MAX PLANCK INSTITUTE
OF ANIMAL BEHAVIOR

Modeling Predator-Prey Interactions from Real Fish Videos: A Generative Adversarial Imitation Learning Approach

Nicole Kloss

Matriculation number: 01/1478633

M.Sc. Social and Economic Data Science

Department of Computer and Information Science

Chair of Cyber-Physical Systems

Supervisor 1: Prof. Dr. Heiko Hamann

Supervisor 2: Prof. Dr. Daniel Keim

External Supervisor: Dr. Liang Li

<https://github.com/nickikloss/Predator-Prey-Thesis>

Konstanz, September 2026

ABSTRACT

Predator-prey interactions are a typical phenomenon in nature, and survival pressure alone can give rise to collective movement in such systems. However, modeling this dynamic is difficult because predator and prey each exhibit role-specific behaviors, changing the environment that the other learns in. This thesis investigates whether predator-prey interactions, particularly pursuit and escape, can be recovered from real video recordings using imitation learning. A hand-labeled 16-prey dataset was constructed and combined with existing 32-prey data through a group-size-agnostic tensor representation, and training data was progressively restricted from general interactions to successful attacks alone to concentrate the target behavior. A Generative Adversarial Imitation Learning framework is implemented in which predator and prey are trained co-evolutionarily, each optimized as separate, modular subproblems using evolutionary strategies. A new policy network and a joint discriminator for fine-tuning the existing architecture were also explored. Restricting training to successful attacks produced the best convergence of any configuration tested, using fewer training windows than the full interaction dataset, indicating that the density of the target behavior in the training data matters more than its volume. Behavioral metrics for pursuit, evasion, and swarm structure showed that the learned models reproduce these behaviors over the time span covered by the labeled clips, with GAIL and behavioral cloning performing similarly overall. Policy maps further revealed that predator-specific structure is encoded directly in the network. Remaining limitations include the assumption of constant movement speed and the absence of species-specific attack dynamics, pointing to clear directions for future work.

TABLE OF CONTENTS

ABSTRACT	ii
LIST OF FIGURES	vi
CHAPTER 1 : Introduction	1
CHAPTER 2 : Literature Review	3
2.1 Self-Organization in Animal Groups	3
2.2 Collective Behavior under Predation	4
2.3 Imitation Learning	6
2.3.1 Behavioral Cloning	6
2.3.2 Inverse Reinforcement Learning	7
2.3.3 Adversarial Imitation Learning	8
2.4 Approaches to Imitating Collective Behavior	9
CHAPTER 3 : Data Collection and Processing	12
3.1 Data Collection	12
3.2 Data Processing	14
3.2.1 Compiling Attack and Interaction Clips	14
3.2.2 Hand-Labeling	15
3.2.3 Identity Assignment and Trajectory Validation	17
3.2.4 Feature Engineering and Tensor Construction	18
CHAPTER 4 : Generative Adversarial Imitation Learning	23
4.1 Policy	24
4.1.1 Pairwise Interaction Network	25
4.1.2 Attention Network	26
4.2 Transition Encoder	26

4.2.1	Encoder Design	27
4.2.2	Encoder Training Using Self-Supervised Learning	29
4.3	Discriminator	31
4.3.1	Discriminator Design	31
4.3.2	Wasserstein Loss with Gradient Penalty	32
4.4	Environment	33
4.5	Evaluation Metrics	33
4.6	Training Procedure	35
4.6.1	Joint Discriminator for Fine-Tuning	36
4.6.2	Training Stabilization	37
CHAPTER 5 : Training Results		39
5.1	Behavioral Cloning	40
5.2	Predator-Prey Policy Learning from Synthetic Data	41
5.3	Predator-Prey Policy Learning from Biological Data	46
5.4	Predator-Prey Policy Learning from Only Successful Attacks	51
5.5	Fine-Tuning with a Joint Discriminator	56
CHAPTER 6 : Evaluation of Learned Predator-Prey Behavior		59
6.1	Pursuit and Evasion	59
6.1.1	Predator Pursuit	60
6.1.1.1	Nearest-Prey Distance	60
6.1.1.2	Pursuit Alignment	61
6.1.1.3	Closing Speed	61
6.1.2	Prey Evasion	62
6.1.2.1	Distance to Predator	63
6.1.2.2	Escape Alignment	63
6.1.2.3	Prey Reorientation	64
6.2	Collective Motion within Swarm	66

6.2.1	Polarization	66
6.2.2	Degree of Sparsity	67
6.2.3	Angular Momentum	68
6.3	Analysis of Policy Networks	70
6.3.1	Predator Reacting to Prey PIN and AN Maps	71
6.3.2	Prey Reacting to Prey PIN and AN Maps	71
6.3.3	Prey Reacting to Predator PIN and AN Maps	72
CHAPTER 7 : Discussion		74
CHAPTER 8 : Conclusion		78
APPENDIX		I

LIST OF FIGURES

3.1	Tank configuration before and after partition removal.	14
3.2	Sample frame during the annotation process in Label Studio.	17
3.3	State definition for focal fish (Yapei Wu et al., 2024).	21
4.1	Overview of the GAIL architecture (Wirtheim & Kloss, in preparation).	23
4.2	PIN and AN with the prey feature vector as input.	24
4.3	VICReg architectural overview (Bardes et al., 2021, p. 2).	30
5.1	Behavioral cloning pre-training loss curves for predator and prey.	40
5.2	Behavioral cloning rollouts with 16 prey.	41
5.3	VICReg pretraining losses for the Couzin predator-prey transition encoders.	43
5.4	Discriminator scores and score difference over generations for the Couzin predator-prey model.	44
5.5	ES update step size and reward difference standard deviation over generations for the Couzin predator-prey model.	45
5.6	MMD and Sinkhorn distances over generations for the Couzin predator-prey model.	46
5.7	VICReg pretraining losses for the video predator-prey transition encoders	47
5.8	Discriminator scores and score difference over generations for the video predator-prey model.	48
5.9	ES update step size and reward difference standard deviation over generations for the video predator-prey model.	49
5.10	MMD and Sinkhorn distances over generations for the video predator-prey model.	50
5.11	GAIL rollouts with 16 prey.	51
5.12	Behavioral cloning rollouts with 16 prey, focusing only on attack data.	52
5.13	VICReg pretraining losses for the video predator-prey transition encoders, focusing only on attack data.	53
5.14	Discriminator scores and score difference over generations for the video predator-prey model, focusing only on attack data.	54
5.15	ES update step size and reward difference standard deviation over generations for the video predator-prey model, focusing only on attack data.	55
5.16	MMD and Sinkhorn distances over generations for the video predator-prey model, focusing only on attack data.	56
6.1	Predator distance to nearest prey model comparison.	60
6.2	Distribution of pursuit alignment model comparison.	61

6.3	Closing speed model comparison.	62
6.4	Distance to predator model comparison.	63
6.5	Escape alignment model comparison.	64
6.6	Prey reorientation model comparison.	65
6.7	Polarization model comparison.	67
6.8	Degree of sparsity model comparison.	68
6.9	Angular momentum model comparison.	69
6.10	Predator response to prey PIN and AN maps.	71
6.11	Prey response to other prey PIN and AN maps.	72
6.12	Prey response to the predator PIN and AN maps.	73
A.1	Reconstructed trajectories for all 27 hand-labeled 16-prey attack clips.	I
A.2	Overview of hyperparameters used for the Couzin predator-prey model.	II
A.3	Overview of hyperparameters used for the video predator-prey model and the attack-only video predator-prey model.	III
A.4	Python libraries used in this thesis.	IV

Introduction

Coordinated movement in animal groups can arise from purely local interactions, without centralized control or global communication. The first models of this collective motion used hand-crafted interaction rules (Reynolds, 1987; Couzin et al., 2002; Vicsek et al., 1995). However, such rules cannot reflect how real animals respond to their surroundings, including external pressures such as those in predator-prey interactions (Wang et al., 2024). Consequently, recent work has increasingly adopted imitation learning approaches to learn collective behavior from observed trajectories. Despite this progress, most existing models focus on homogenous groups (Heras et al., 2019; Yu et al., 2021; Schafer et al., 2022; Yapei Wu et al., 2024), and several train on simulated rather than observed behavior (Yu et al., 2021; Yapei Wu et al., 2024). Further, although survival pressure can give rise to collective motion in predator-prey systems (Li et al., 2023), only a limited number of studies address these settings (Song et al., 2018; Yifan Wu et al., 2025). Learning the interactions between predator and prey from real animal recordings therefore remains underexplored, even though predation directly shapes how animal groups move. Modeling it is difficult, because predator and prey pursue opposite goals, so each role’s behavior changes the environment the other learns in, and attacks are rare and brief in real recordings.

Wirthheim (2026) began exploring this topic by developing a framework in which predator and prey policies are trained together through adversarial imitation on real fish recordings, with the aim of reproducing pursuit and evasion. His prey-only policy converged, but training on the full predator-prey system with real video data converged only partially and the model did not capture the intergroup dynamics. The present thesis builds on this work to determine whether these interactions can be recovered from real video recordings at all, and whether the limitation lies in the data, the policy architecture, or the training objective.

To keep the results comparable with Wirthheim’s, several components of his framework serve as a starting point for this thesis, including the generative adversarial imitation learning (GAIL) framework, its stabilization measures, his 32-prey data, and the six training metrics. Building on this, the thesis makes four central contributions. First, it introduces a new dataset of 27 successful attacks and 40 predator-prey interactions, each with 16 prey. Second, it develops a new data processing pipeline with a representation that is independent of prey group size, using padded neighbor slots and an active mask throughout training, so that a single model can handle different numbers of prey. Third, it investigates which factors limit the learning of predator-prey interactions by trying an alternative policy network, restricting training to successful attacks to reduce dilution of the target behavior, and adding a joint discriminator for fine-tuning on top of the existing architecture. Fourth, it extends the behavioral evaluation by comparing the learned policies with the expert data, a behavioral cloning model, and the Couzin model on pursuit, evasion, and swarm measures.

Of these changes, those to the data had the largest effect. Training on real video data stabilized for both roles when using the full dataset consisting of both the 16- and 32-prey conditions, an improvement over the partial convergence Wirthheim (2026) reported with 32-prey data alone. Restricting training to only successful attacks gave the best distribution match in this work. When evaluated, this attack-only policy reproduced pursuit and evasion over the time span covered by the labeled attacks, though behavioral cloning performed comparably. Beyond biology, such learned policies could also inform controllers for multi-robot systems in which agents take on different roles, similar to what Yapei Wu et al. (2024) demonstrated for homogeneous group robot swarms. The remainder of this thesis has the following structure. Chapter 2 reviews the literature on self-organization in animal groups, predator-prey systems, and imitation learning. Chapter 3 describes how the predator-prey recordings were collected and processed into expert demonstrations. Chapter 4 presents the GAIL framework, in which the predator and the prey each have their own policy and discriminator. Chapter 5 reports the GAIL training results. Chapter 6 evaluates the learned policies using pursuit, evasion, and swarm measures, together with an analysis of the policy networks. Chapter 7 discusses the findings and limitations, and Chapter 8 concludes the thesis.

Literature Review

This chapter details the theoretical and methodological foundations for modeling collective behavior in predator-prey systems using imitation learning. It begins with self-organization in animal groups, outlining how local interaction rules can generate coordinated motion without central control. It then turns to predator-prey systems, where heterogeneous roles with differing objectives make the dynamics fundamentally different from homogeneous collectives. Next, imitation learning is introduced as a framework for learning behaviors from observations. Finally, existing data-driven approaches to imitating collective behavior are detailed, emphasizing that most work focuses on homogeneous groups, leaving the learning of role-asymmetric predator-prey interactions from real video data largely unaddressed.

2.1. Self-Organization in Animal Groups

A school of thousands of fish can shift direction almost simultaneously, with no leader, no signal given to the entire group, and no individual aware of more than its immediate surroundings, yet local perception alone is enough to sustain the collective motion (Reynolds, 1987; Couzin et al., 2002). This is a well-known example of self-organization, which refers to order that emerges from the interactions between a system's components without external coordination (Sumpter, 2006; Di Marzo Serugendo et al., 2003).

Self-organization of this kind was first formalized by Reynolds (1987) in the Boids model, which reduced flocking to three local interaction rules. The first is alignment (or orientation), where individuals adjust their movement to match that of their neighbors. The second is cohesion (or attraction), which draws individuals close together to keep the group intact. The third is separation (or repulsion), which prevents collisions by maintaining distance from neighboring individuals.

These three rules alone were sufficient to reproduce the flocking of birds, without the need for explicit communication, global coordination, or leadership. Vicsek et al. (1995) built on this idea, showing that simple alignment rules among particles could produce large-scale ordered motion. Couzin et al. (2002) extended this framework further by modeling fish schooling, applying the interaction rules hierarchically across three zones. The innermost is the repulsion zone, which is the highest priority and where individuals keep a minimum distance from others at all times. Beyond it is the orientation zone, in which headings are matched to those of nearby fish. Farthest out is the attraction zone, which pulls a fish back toward the rest of the group. These agent-based models successfully produce basic swarm patterns and are simple to interpret, showing that coordinated dynamics can emerge from local interaction rules.

However, they remain static and fail to capture the intricacies of a real biological system, since they rely on assumed interaction rules rather than measured ones (Wang et al., 2024). Real animal collectives must also adapt dynamically to changing environments and external pressures, which these models do not account for. For instance, in predator-prey systems, coordinated movement can emerge from survival pressure alone, without any explicit alignment rule (Li et al., 2023). In the reinforcement learning setup of Li et al. (2023), agents were trained with a survival reward without cohesion or explicit alignment terms, yet prey still developed flocking and swirling, while predators developed confusion tactics and marginal predation. Predation is therefore not a disturbance to an otherwise self-organized group, rather it is part of what generates the collective behavior in the first place.

2.2. Collective Behavior under Predation

Predator-prey systems break an assumption shared by the models above, that every individual in the collective follows the same behavioral strategy (Reynolds, 1987; Couzin et al., 2002; Vicsek et al., 1995). They are a specific form of multi-agent systems, in which multiple agents interact so that each agent’s behavior both shapes and is shaped by the actions of the others (Yong and Miikkulainen, 2001). What distinguishes predator-prey systems from other multi-agent settings is that the interacting groups are heterogeneous and the two groups pursue opposing objectives.

The predator chases the prey and attempts to strike, while the prey try to evade being captured. Pursuit and evasion are the main behaviors that characterize predation (Peterson et al., 2021), and the resulting dynamics are driven by survival rather than by coordination alone.

The two roles behave in characteristically different ways. Within the prey group, individuals behave cooperatively, and synchronized directional changes can confuse an approaching predator. Once a predator is detected, the prey form a compact swarm, which increases their survival chances (Magurran & Pitcher, 1987; Li et al., 2023), and while escaping, they collectively attempt to maximize their distance from the predator (Peterson et al., 2021). The strength of this collective response is not fixed, however. Calovi et al. (2015) showed in a data-driven school model that a group's susceptibility to perturbation depends on the collective state it currently occupies, so the same disturbance can produce a strong coordinated response or almost none. Predators, in contrast, typically employ dispersion tactics. During an attack, the predator aligns its heading with the prey (Soto et al., 2020) and swims towards the center of a prey swarm, breaking its cohesion, before focusing on individuals that become isolated as a result.

These role-specific behaviors are what make predator-prey systems difficult to imitate. Collective dynamics do not only arise from homogeneous interactions, but also emerge from the continuous exchange between predator and prey during pursuit and evasion. Because each group's behavior forms part of the environment the other responds to, neither can be modeled independently of the other. What has to be reproduced is therefore the interaction itself, pursuit, avoidance, and escape, which is a property of the relationship between the groups rather than of either group's motion on its own. Accordingly, recent advancements in computational power and imitation learning methods have prompted a shift in focus from specific hand-crafted interaction rules to data-driven models. Such approaches offer a way to capture predator-prey dynamics from real-world observations, accommodating the heterogeneity and coupling that rule-based models cannot.

2.3. Imitation Learning

Imitation learning and reinforcement learning (RL) share a common framework in which behavior is modeled as a Markov Decision Process (MDP). In an MDP, an agent makes a sequence of decisions, and the outcomes are partly stochastic and partly determined by the agent’s own actions (Pierson and Kao, 2025).

An MDP is defined by:

- a set of states $\mathbf{S}$,
- a set of actions $\mathbf{A}$,
- a transition model $\mathbf{P}(\mathbf{s}'|\mathbf{s},\mathbf{a})$ giving the probability of moving to state $\mathbf{s}'$ after taking action $\mathbf{a}$ in state $\mathbf{s}$, and
- a reward function $\mathbf{R}(\mathbf{s},\mathbf{a})$.

The goal is to find an optimal policy π^* that maximizes the expected cumulative reward. RL solves this in a forward direction by deriving the optimal policy from a known reward. However, it is difficult to apply this framework to animal behavior, because what the animal is optimizing is unknown. Imitation learning avoids this issue because it learns behavior directly from expert demonstrations instead of explicitly defined reward functions (Hussein et al., 2017). Existing approaches to imitation learning include behavioral cloning (BC), inverse reinforcement learning (IRL), and generative adversarial imitation learning (GAIL). These algorithms differ in their underlying assumptions, and those differences determine which of them is suited to the system studied here.

2.3.1. Behavioral Cloning

BC is a foundational approach to imitation learning as it offers a simple way to learn policies directly from expert data through supervised learning (Torabi et al., 2018). Using state-action pairs from the expert, the goal is to make the learned policy’s actions as close as possible to the expert’s actions, with no interaction with the environment required during training. However, BC’s weakness is that it makes the assumption that state-action pairs are independently and identically distributed samples. In reality, the state-action pairs in an MDP are sequentially created. This

means that the action taken at one state impacts the state at the next, so consecutive samples are neither independent nor identically distributed, rather they are temporally correlated (Ross et al., 2011). In practice, small prediction errors build up over time, which pushes the agent into states that the expert never reached and that were not present in the training data. When the agent ends up in such unseen states, there is no corrective signal, meaning no guidance to correct its actions, so the errors compound further. As a result, BC requires a large amount of demonstration data, since the agent can only act reliably in states the expert has already visited. Yapei Wu et al. (2024) demonstrated this empirically, training a BC baseline on the same network architecture and demonstration data as their adversarial method. They found that BC required substantially more demonstrations to reach comparable performance and degraded sharply when given less, which the authors attribute to its greater susceptibility to compounding error. BC is therefore valuable as a pretraining step for adversarial methods and as a baseline for comparison, but it is not suited as the primary method here, where behavior must remain stable over extended rollouts.

2.3.2. Inverse Reinforcement Learning

While RL tries to learn the best policy that maximizes an explicitly known reward, IRL assumes the reward is implicit in the expert demonstrations and infers it from them (Arora and Doshi, 2020). Concretely, IRL looks for the reward function that best explains the demonstrated behavior, removing the need to specify one manually, and the imitation policy is then obtained by optimizing that recovered reward (Torabi et al., 2018). Though IRL avoids the compounding error issue that came about in BC, different issues are introduced. The first is ambiguity (Arora and Doshi, 2020). Since many reward functions can explain the same demonstrations equally well, the reward inference becomes underdetermined. The second is high computational complexity (Arora and Doshi, 2020). Recovering the reward requires the forward decision problem to be solved repeatedly, once per iteration of the inference, so the computational cost grows disproportionately with the size of the state and action spaces. In a biological environment where each additional agent enlarges that space, IRL becomes difficult to scale to systems of this complexity. Both difficulties have been addressed in literature. Yu et al. (2021) addressed the ambiguity by restricting the learned function to a reward estimator and a potential shaping term, which permits the underlying reward to be

recovered under suitable assumptions. Schafer et al. (2022) addressed the computational expense by formulating movement as a linearly-solvable MDP, for which the optimal policy has an analytic solution. State costs can be obtained by transforming the estimated costs-to-go, rather than by re-solving the forward problem at every iteration. These are effective responses, but IRL still recovers a reward rather than a behavior. Reproducing the demonstrations requires a further step, in which a policy is optimized against the recovered reward, and any error in that step carries through to the behavior. Since the aim here is to reproduce observed interaction patterns rather than to explain why the animals behave as they do, the reward is a detour that can be skipped.

2.3.3. Adversarial Imitation Learning

Generative adversarial imitation learning (GAIL) (Ho and Ermon, 2016) adapts the generative adversarial networks (GANs) of Goodfellow et al. (2014) to an imitation learning setting. The policy π_θ acts as the generator, producing trajectories by acting in the environment. The discriminator D_β is trained to separate the state-action pairs in those rollouts from the pairs in the expert demonstrations. The goal of the policy is to fool the discriminator and make the two indistinguishable. The policy and discriminator are optimized in alternation (Ho and Ermon, 2016). After every policy update, newly generated trajectories are paired with the demonstrations to retrain the discriminator, which returns the probability that a given state-action pair came from the expert. The discriminator is optimized through a binary cross-entropy objective, and its output is then used as a learned reward for the policy, rising as the policy produces pairs the discriminator judges to be expert-like. At the optimum the discriminator can no longer separate the two sources.

GAIL has several advantages over BC and IRL. First, compounding errors are avoided, since learning does not depend on supervised action labels but on trajectories the policy generates itself. Second, training is faster and more sample-efficient than IRL, because no reward function has to be recovered, which means no assumptions need to be made about what the animals are optimizing. Third, and most importantly, GAIL matches distributions directly. Rather than optimizing a proxy, it asks whether the distribution of states and actions produced by the policy matches the expert's distribution, and learning is successful once the two coincide. This matching does not depend on the

number of agents involved, so the approach extends naturally to multi-agent systems. It also places no constraint on the number of discriminators, so roles with different behavioral strategies can each be assigned their own, as proposed by Song et al. (2018) for competitive settings. However, these advantages come at the cost of stability, since training a policy and a discriminator simultaneously introduces its own difficulties. If the discriminator learns too quickly, the reward signal degenerates and the policy can no longer adapt. In a predator-prey setting this is especially amplified, since two policies are trained at once and any improvement in one changes the environment the other is learning in. GAIL is also susceptible to mode collapse, where the generator produces only a subset of the behaviors that the discriminator favors, causing behavioral diversity to decrease (Ho and Ermon, 2016). The policy then stops exploring the full range of behavior present in the expert demonstrations. Methods for stabilizing training are introduced in Chapter 4.

2.4. Approaches to Imitating Collective Behavior

The application of imitation learning to biological collective behavior is represented in literature, with a variety of data-driven approaches that learn about collective behavior from observed trajectories. More common in the literature is imitation of homogeneous collectives, where all agents follow the same behavioral strategy. Within this scope, a few of the approaches learn from synthetic demonstrations. Yapei Wu et al. (2024) generated milling, schooling, and swarming with the Couzin model and used adversarial imitation learning to imitate the behavior. Specifically, they used a pairwise interaction network and an attention-based aggregation mechanism previously proposed by Heras et al. (2019) to model interactions and learn a global policy, and they managed to transfer the learned policy to a swarm of 20 wheeled robots. Yu et al. (2021) used demonstrations from the Vicsek model with a parameter-sharing adversarial inverse reinforcement learning framework to recover a shared policy along with the underlying reward function from collective demonstrations. Using synthetic demonstrations is useful for understanding how the environment is set up and for gaining insight into the overall system, but it limits what the model can learn to what is explicitly put into the simulation. Accordingly, the following studies use real video recordings of fish as their primary data source. In each case, the recordings are first passed through automatic

multi-object tracking, which reduces the video to one trajectory per fish, carrying its heading, speed, acceleration, and distance to each neighbor. The individual trajectory can then be used for learning. Schafer et al. (2022) applied Bayesian IRL to recover the cost structure underlying the movement of a captive guppy population. By formulating movement as a linearly-solvable Markov decision process, they were able to recover the latent decision costs where the observed trajectories would be optimal under the recovered cost function. Their Bayesian approach yields a distribution over possible costs, which provides smoothing and uncertainty estimates instead of a single point estimate. Heras et al. (2019) trained two modular networks on the extracted expert trajectories. One network models the pairwise influence of each neighbor on a focal individual, while the other aggregates these influences across the swarm using an attention mechanism that weights neighbors according to spatial context such as positions, velocity, and heading angles, which come from the trajectories. From this representation, the focal individual’s turning decisions are predicted by the supervised learning model.

So far, the reviewed methods have focused on homogeneous collectives with a shared behavioral strategy across all agents. However, predator-prey systems where heterogeneous groups with different roles interact have far less literature available. In these situations, multiple agents act simultaneously, and their positions and actions jointly affect what the focal agent perceives, meaning that the environment never settles (Yapei Wu et al., 2024). As noted in Section 2.1, Li et al. (2023) showed that survival pressure alone can drive swarming, with predators and prey co-adapting their policies based on opposing rewards. These behaviors were generated by RL rather than imitated from observation, so the result shows what could produce such dynamics rather than what real animals do. Song et al. (2018) provided the theoretical foundation for multi-agent imitation learning by proposing a GAIL framework with separate discriminator-policy setups for cooperative, mixed, and competitive interactions. However, they tested it only on standard Markov game benchmarks, not on real biological data. Yifan Wu et al. (2025) removed the tracking step, training a masked video autoencoder on raw footage and using the resulting latent states as the target for adversarial imitation, so that the policy matches the distribution of motion patterns rather than reconstructed trajectories. This avoids the occlusion and overlap problems of tracking-based pipelines. They

reproduced circling, alignment, aggregation, cohesion, feeding and intraspecific chasing, with a predation clip existing in their video corpus, but predation is given neither a behavior-specific reward nor a place among their evaluated results.

Imitating collective behavior is therefore more complex in predator-prey systems. Data-driven approaches have made advancements beyond hand-crafted rules, but they are often used with homogeneous groups where every individual pursues the same objective. Wirtheim (2026) addressed this by building a role-asymmetric adversarial imitation framework with separate policies and discriminators for predator and prey, co-trained on both simulated and real video data, and evaluating inter-group behavior through predator-prey distance and escape alignment. His learned policies reproduced basic group motion but did not reliably reproduce pursuit or coordinated avoidance. What is still open is whether predator-prey interactions can be recovered from real video recordings at all, and where the bottleneck lies: in the data, the policy architecture, or the training objective. This thesis addresses this gap.

Data Collection and Processing

This chapter describes the data collection and processing methods used to transform raw predator-prey interaction videos into input tensors for imitation learning. It begins by introducing the experimental design under which the predator-prey interactions were recorded. Then, it describes in detail the pipeline that converts these raw recordings into a relative-state representation of the interactions between individuals, which forms the expert data for model training. This processing pipeline comprises clip selection, hand-labeling, identity assignment to build trajectories, feature engineering, and tensor construction. The 16-prey dataset produced by this pipeline, together with the group-size-agnostic tensor representation that makes it usable, constitutes one of the central contributions of this thesis.

3.1. Data Collection

The predator-prey video recordings were made by Li’s research group in the Department of Collective Behavior at the Max Planck Institute of Animal Behavior (L. Li, 2025). Since the experimental setup and data collection were carried out before the work on this thesis began, the subsequent section is based on the research group’s original description.

The prey species is *Leucaspis delineatus* (sunbleak), a freshwater fish that forms swarms near the surface of still or slow-flowing waters (European Commission, 2026b; Leu et al., 2009). Sunbleaks are omnivorous, feeding on algae, insect larvae, and mosquitoes, and originate from Central and Eastern Europe. The predator species is *Esox lucius* (northern pike), which typically inhabits reed-dominated, vegetated zones close to the shoreline in freshwater. They are predominantly piscivorous and often prey on weak or sick fish, including conspecifics, thereby helping to maintain a healthy

aquatic ecosystem (European Commission, 2026a). Northern pike are found in Northern and Central Europe, as well as parts of Asia and North America.

The study followed a common pipeline across all the group's experiments: individual accommodation and acclimatisation, collective behavioral trials manipulating group size and predation risk, and video-based extraction and analysis.

Wild-caught sunbleaks and pike were transported to a climate-controlled laboratory and housed in large holding tanks, which contained artificial vegetation and covered areas. The sunbleaks and pike were kept in separate rooms to prevent visual contact outside of trials, and all fish acclimated to the laboratory conditions for one week before the trials began. Trials were conducted in a 1.5 × 1.5 m featureless white tank, positioned in an isolated room and surrounded by black curtains to minimize visual and acoustic disturbance. Two layers of carpet were laid beneath the tank to dampen vibrations. The tank was filled to a water depth of 6 cm, and the water temperature, conductivity, and pH were kept at levels comparable to those in the species' home tanks (approximately 16.7°C, 700 S, and pH 7.1). To increase predatory motivation, pike were not fed during the 24 h preceding each trial.

During collective behavioral trials, groups of sunbleaks of different sizes (1, 2, 4, 8, 16, 32, or 64 individuals) were netted from their home tanks and placed in a covered bucket for transfer to the center of the tank. A pike was then moved into a partitioned corner of the tank using the same handling procedures. The lower section of the partition was opaque to block visual contact between predator and prey at the start of the trial. This entire transfer process required around 3 to 5 minutes. Once all sunbleaks were introduced, the curtains around the tank were closed, the video recording started, and the experimenter moved away from the tank. After 5 minutes, the partition was lifted using a pulley, giving pike access to the whole tank (Figure 3.1). Recording continued for a further 10 minutes, after which a low-stress removal protocol was applied: the main lights were switched off, a red light was switched on for 1 minute, the curtains were opened, and the sunbleaks were slowly netted back into their home tanks, followed by the pike. Each group-size condition required around 1 hour per trial, and experiments were limited to a maximum of 10 trials per 24 h.



Figure 3.1: Tank configuration before and after partition removal.

Fish behavior was recorded using an overhead array of four cameras positioned roughly 2 m above the tank. The videos were captured at $\sim 24\text{--}90$ frames s^{-1} with a spatial resolution of 2048×2048 pixels, and cameras were connected to an external recorder, which saved the data directly to a solid-state drive. Once all experiments had been completed, the fish were temporarily kept in social holding tanks before being transferred to large mesocosms at the Limnological Institute for use in further behavioral studies.

3.2. Data Processing

The aim of data processing is to turn raw video recordings into tensors that capture predator-prey dynamics and can be used for model training. The present study focuses on trials with 16 and 32 prey individuals. For the 32-prey condition, pre-processed data from Wirtheim (2026) was used with minimal modifications, comprising 32 attack and 35 interaction clips. For the 16-prey condition, no processed data existed, and a new pipeline was developed for this thesis that is agnostic to the size of the prey group. The pipeline consists of selecting and clipping attack and interaction episodes, hand-labeling fish positions, recovering individual identities with the Hungarian algorithm, and assembling padded, windowed predator-prey tensors from the resulting relative features.

3.2.1. Compiling Attack and Interaction Clips

The total duration of the 16-prey trial videos is 10 h 59 min 30 s. This footage was reviewed in full to locate the clips of interest. The majority of this footage shows the predator swimming along the

tank wall or remaining largely stationary while the prey school without much apparent regard for the predator. Successful attacks and meaningful predator-prey interactions occur only sporadically. Meaningful interactions were taken to include failed attacks, active pursuit and evasion events, and instances in which the predator’s movement caused the prey to behave differently from how they typically would when schooling. As this thesis focuses on inter-group dynamics, only successful attacks and meaningful interactions were clipped and processed. This selective clipping is further justified by previous work demonstrating that prey-only schooling behavior can be learned through imitation learning (Wirtheim, 2026), making predator-prey interactions the novel component of this study.

After reviewing all 16-prey videos, 27 successful attacks were identified and clipped to a standard 5-second window, resulting in a total of 2 min 15 s of attack footage. This 5-second window usually captures the entire attack because, shortly after a successful attack, the predator tends to cease movement and the prey scatter in a more erratic and unstructured way, consistent with observations in the literature (Marras & Domenici, 2013). An additional 40 interactions were identified and clipped to lengths ranging from 5 seconds to 1 minute, resulting in a total of 10 min 5 s of interaction footage. Together, these 67 clips contain 12 minutes and 20 seconds of behaviorally dense footage, selected from nearly eleven hours of total recordings in which predator-prey interactions are mostly absent. In total, these clips make up 1.87% of the 16-prey trial videos. This curation does not reduce the available data, rather, it concentrates it on the interactions this thesis aims to model.

3.2.2. Hand-Labeling

The clipped attack and interaction videos featured fast and chaotic movement, causing disturbances to the surface of the water and often resulting in fish swimming underneath each other. An initial attempt at automatic tracking was explored, following Wirtheim (2026), combining a YOLO model, which is well-established for object detection tasks (Ultralytics, 2026b), with DeepSORT, a multi-object tracking algorithm (Wojke, 2023). However, this automatic approach struggled under fast and chaotic motion, often missing prey, leading to inconsistent identity assignment across frames

and yielding very few usable 10-frame windows. Detection and identity assignment perform poorly precisely during rapid acceleration, tight turning, and occlusion, which are the defining characteristics of a strike and the corresponding evasive response. As a result, an automatically tracked dataset is not just smaller; it is systematically biased against the very behavior this thesis aims to study. Since attack and interaction footage comprised only 1.87% of the total 16-prey recordings, hand-labeling was feasible and was used instead to maximize accuracy and ensure that all pursuit and evasion behaviors were reliably represented in the expert data used for model training.

The 16-prey trials used in this thesis were recorded at 25 fps. For hand-labeling, every second frame was extracted from the clipped footage as JPEGs, giving an effective sampling interval of $\Delta t = 0.08$ s between the labeled frames, which means 12.5 labeled frames per second. This decision was made because every frame requires 17 keypoints, one for each of the 16 prey and one for the predator, to be placed by hand, so halving the labeling rate doubled the amount of footage that could be annotated. A five-second attack clip using every other frame still consists of 63 frames to label, which was found to be sufficient to follow a successful attack. For the meaningful interactions, which vary in length, the number of frames to label ranged from 63 to 750.

Frames were uploaded to *Label Studio* (Tkachenko et al., 2020–2025) for manual annotation of predator and prey positions (Figure 3.2). After approximately 300 frames were labeled, a YOLOv8 convolutional neural network model (Ultralytics, 2026a) was trained on these annotations in order to pre-label the remaining footage and reduce the amount of manual labor. Unlike in the automatic pipeline above, this model is used only as a labeling aid. For training the model, the existing Label Studio annotations were converted into Ultralytics YOLO pose format, and the labeled attacks were split into training and validation sets, keeping all frames from the same attack together to avoid data leakage. The model was trained for up to 100 epochs with early stopping. After applying the trained model to the remaining frames in *Label Studio*, every frame was manually checked to verify that all label positions were correct and that the expected number of fish was present. Each labeled attack or interaction was then exported as a JSON file containing the pixel position of every fish in every labeled frame. Of these, all 27 attack clips and 8 of the interaction clips were hand-labeled

for this thesis, each with a consistent number of 16 prey and 1 predator in every frame. Across the 27 attack clips alone, this amounts to 1,701 labeled frames and approximately 29,000 individual fish positions, each placed or verified by hand. The remaining interaction clips have been compiled and clipped, and are available for future work.

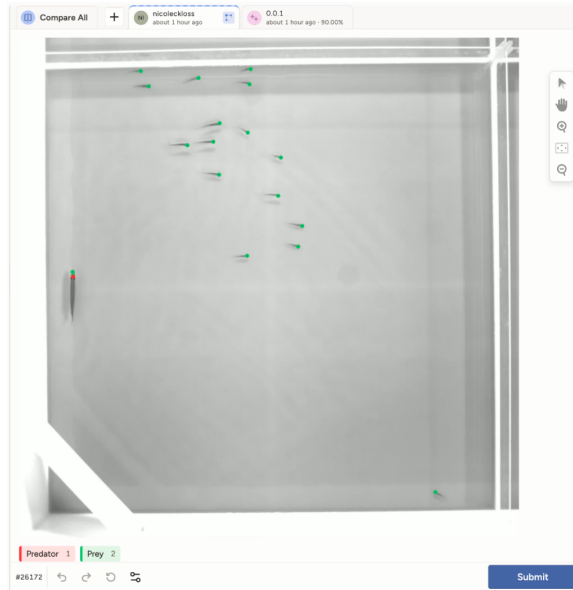


Figure 3.2: Sample frame during the annotation process in Label Studio.

3.2.3. Identity Assignment and Trajectory Validation

Before identities can be assigned, the labeled coordinates must be placed on a common spatial scale. Because the spatial resolution of the videos was slightly altered during clipping, the annotated position of each clip was first rescaled to a common 2160×2160 canonical frame to ensure a consistent spatial scale across clips. The positions were then normalized to the range $[0, 1]$ by dividing by the size of the canonical frame. This keeps the input scale independent of the recording resolution and improves training stability.

The hand-labeled export derived in Section 3.2.2 provides the pixel locations of all the fish within each frame, but no information on which fish corresponds to which in the next frame. The Hungarian algorithm (Kuhn, 1955) was therefore applied to recover individual continuous trajectories for every fish across frames. For each pair of consecutive frames, a cost matrix was created using pairwise

Euclidean distances between all detections in the previous and current frames, and the algorithm identified the one-to-one assignment that minimized the total distance summed over all matched pairs. Predator and prey were matched separately, with the predator labeled as its own class since there is only one predator per clip. Consequently, the predator trajectory can be treated as reliable, as no identity switch is possible.

To ensure the quality of the prey trajectories, the displacement of each fish between consecutive frames was measured. An accurate assignment implies that a fish moved a plausible distance within 0.08 s, whereas an unusually large displacement indicates that an identity was assigned incorrectly. In 24 of the 27 attack clips, the largest distance any fish moved between two labeled frames ranged from 40 to 90 pixels. The remaining 3 clips had maximum distances way beyond this range: 305 pixels in *pred_attack_17_16*, 1273 pixels in *pred_attack_23_16*, and 511 pixels in *pred_attack_24_16*. No clip contains a step between 90 and 300 pixels, so the two groups are clearly separated. In those 3 clips, 99% of all movements remain below 45 pixels, and after a closer inspection, the outliers were traced to only 3 individual frame annotations. So, the affected frames were re-annotated, and the pipeline was rerun. After correction, the largest per-step displacement across all 27 attack clips is 89 pixels, and no identity switches were detected. The same check was applied to the interaction clips, where no outliers were found. Secondly, the reconstructed trajectories were visually validated by comparing them with the original corresponding video clips. They were manually inspected to make sure the identities were consistent throughout the clip and that the paths matched those in the footage. The reconstructed trajectories for all 27 attack clips can be seen in Figure A.1 in the Appendix. Together, these checks confirm that the processed trajectories are reliable and ready for model training.

3.2.4. Feature Engineering and Tensor Construction

The state representation used in this thesis is based on the work of Wirthheim (2026), which is consistent with the evidence reported by Heras et al. (2019) and adopts the state representation of Yapei Wu et al. (2024). It is constructed using the distances and relative velocities between individuals rather than their absolute positions. Keeping this representation was a deliberate choice to ensure

that the tensors created from this pipeline remain comparable to previous work. Any difference in model performance can therefore be attributed to the model and data changes introduced in this thesis, rather than to a change in the input. The extensions made at the tensor level, specifically the padded neighbor dimension with an active mask, are detailed in this section.

With the validated trajectories, the expert tensors can now be constructed. For every fish in a frame, and for every one of its neighbors, the tensor stores a small set of values describing that neighbor from the focal fish's point of view (Figure 3.3). These values consist of where the neighbor is relative to the focal fish (dx , dy), how the neighbor is moving relative to the focal fish (vx , vy), and the focal fish's own turn at that timestep ($\Delta\theta$), which is repeated across all of that fish's neighbors, since the turn is specific to the fish. Additionally, a binary feature marking whether a slot holds a real fish or padding is added to both the predator and prey tensors, and a binary feature marking whether that neighbor is a predator is added to the prey tensors.

From the trajectories, the velocity of each fish was computed as the difference between its position in one labeled frame and its position in the next, giving both how far the fish moved and in which direction. Since only every second frame was annotated, this difference was divided by two so that velocity is expressed per video frame. The direction of this movement is taken as the heading angle θ , since a fish is assumed to face the direction it is swimming. The heading is what makes the focal fish's individual point of view possible, because to describe a neighbor from its perspective, it must first be known which direction the fish is facing. From here, positions and velocities can be expressed relative to each focal fish. For a focal individual and a neighbor i , the relative position is the difference between their normalized coordinates:

$$\begin{aligned}x_i &= x_{\text{neighbor}} - x_{\text{focal}}, \\y_i &= y_{\text{neighbor}} - y_{\text{focal}}.\end{aligned}\tag{3.1}$$

The relative velocity is obtained by rotating the neighbor's velocity by the negative heading angle of the focal fish, so that the components describe the neighbor's movement from that fish's point of view:

$$\begin{bmatrix} v_{x,i} \\ v_{y,i} \end{bmatrix} = \begin{bmatrix} \cos(-\theta_{\text{focal}}) & -\sin(-\theta_{\text{focal}}) \\ \sin(-\theta_{\text{focal}}) & \cos(-\theta_{\text{focal}}) \end{bmatrix} \begin{bmatrix} v_{x,\text{neighbor}} \\ v_{y,\text{neighbor}} \end{bmatrix}. \quad (3.2)$$

These relative velocities are clipped to a maximum of 25 pixels per frame and scaled to $[-1, 1]$, so that distortion from extreme values is avoided. The limit was raised from the 10 pixels per frame used in previous work because this thesis focuses on attacks and meaningful interactions, where high-speed frames are more strongly represented. In the 16-prey attack clips, the 95th percentile of the relative speed is 14.5 pixels per frame, and in half of all attack windows, the fastest relative speed exceeds 19.4 pixels per frame. With the previous limit, the peak moment of most attack windows would have been distorted, which is exactly the part of the interaction this thesis aims to model. Therefore, a bound of 25 keeps these values while still removing unrealistic outliers.

The next value stored in the tensor is the action, written as $\Delta\theta$, which is how much a fish turns from one timestep to the next. It is computed as the difference between its heading in one frame and its heading in the next. The difference is wrapped to the interval $[-\pi, \pi]$ to avoid discontinuities and then scaled to $[0, 1]$ to match the output range of the network. The action is used only for pretraining with behavioral cloning. The GAIL discriminator never sees it, since it is trained on state transitions alone. Nevertheless, it is kept in the tensor for convenience in data handling. The last frame of a clip has no following frame and therefore no heading change, so its action is set to zero. The frame is kept rather than dropped, because it still provides state information.

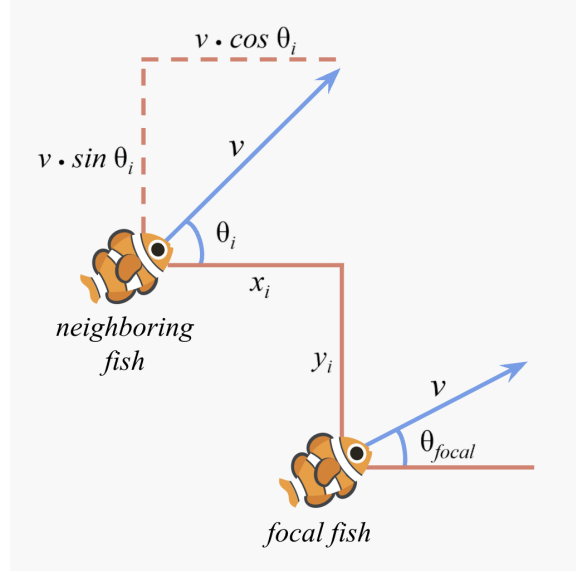


Figure 3.3: State definition for focal fish (Yapei Wu et al., 2024).

Next, the tensors were padded, and an active mask was added. Wirthheim’s (2026) pipeline was built for a fixed group of one predator and 32 prey, with this number hard-coded throughout. The data used in this thesis, however, contains both 16-prey and 32-prey clips, and the aim was to process both conditions with the same network rather than building separate models for the different prey sizes and splitting up the available data. This thesis therefore introduces a padded neighbor dimension with an accompanying active mask. A fixed number of 32 slots is always used, so that in a 16-prey clip, 16 slots hold real fish and 16 are empty. Padding alone is not sufficient, because an empty slot filled with zeros would be numerically the same as a real neighbor located exactly at the focal fish’s position and moving exactly with it. An active column was therefore added to mark which slots are real and which are empty, so that the network can exclude the empty ones. This extension is what makes the new 16-prey data usable alongside the existing 32-prey data within a single model. It also generalizes the pipeline: any prey group size up to 32 prey with one predator, can now be processed without modification, so future work can add further group sizes directly. Lastly, because the predator and prey policies are trained against one another, the prey input must also encode which of its neighbors is the predator. A binary role feature is therefore

appended to each prey-neighbor slot. This predator flag was only included in the prey tensors, so that prey can distinguish between the predator and other prey.

The resulting feature tuples for the two roles are:

$$\text{Predator: } (d_x, d_y, v_x, v_y, a), \quad \text{Prey: } (\text{flag}, d_x, d_y, v_x, v_y, a) \quad (3.3)$$

where $a = 1$ if the slot holds a real fish and $a = 0$ if it is padding.

The role flag is multiplied by the active mask, so that it is only set on real, non-padded slots. The focal fish's own speed is deliberately omitted from the input, as in Wirtheim (2026) and Yapei Wu et al. (2024). All velocity information reaches the network through the relative neighbor components, which keeps the input dimensionality low. Additionally, following their approach, each clip is split into overlapping sequence windows of length ten to train the model.

The resulting tensors have 5 dimensions. The batch dimension (B) is the number of windows processed. The temporal dimension (T) holds the consecutive frames of a window. The agent dimension (A) is the number of focal fish, which is 1 for the predator and 32 for the prey. In a 16-prey clip the agent dimension is padded to 32 in the same way as the neighboring dimension, with the active mask identifying the occupied slots. The neighbor dimension (N) is fixed at 32 slots. The feature dimension (F) holds the values describing one focal fish in relation to one neighbor. This gives the following tensors:

$$\text{Predator Tensor} \in \mathbb{R}^{3,771 \times 10 \times 1 \times 32 \times 6}, \quad \text{Prey Tensor} \in \mathbb{R}^{3,771 \times 10 \times 32 \times 32 \times 7}$$

Applying the pipeline to all four sources, the previously pre-processed 32-prey attack and interaction clips and the newly processed 16-prey attack and interaction clips, yields 3,771 windows. Restricting this to the attack clips alone yields 2,200 windows, which is the dataset used for the results reported in Section 5.4. Both are considerably larger than the 793 windows used in the previous work, giving substantially more training data.

Generative Adversarial Imitation Learning

With the expert tensors extracted from the predator-prey recordings, the next step is to learn a policy using GAIL that reproduces this behavior. This chapter introduces the GAIL architecture, where predator and prey each receive their own policy, transition encoder, and discriminator. Rollouts from each policy are mapped into the same state representation as the expert demonstrations, making the two directly comparable. Role-specific transition encoders and discriminators then judge how closely the generated transitions match the corresponding expert behavior. Based on the discriminator feedback, both policies are alternately optimized, allowing predator and prey to co-adapt through their interactions in the shared environment (Figure 4.1). The remainder of this chapter covers the environment setup, the metrics used to evaluate training performance, and the training procedure.

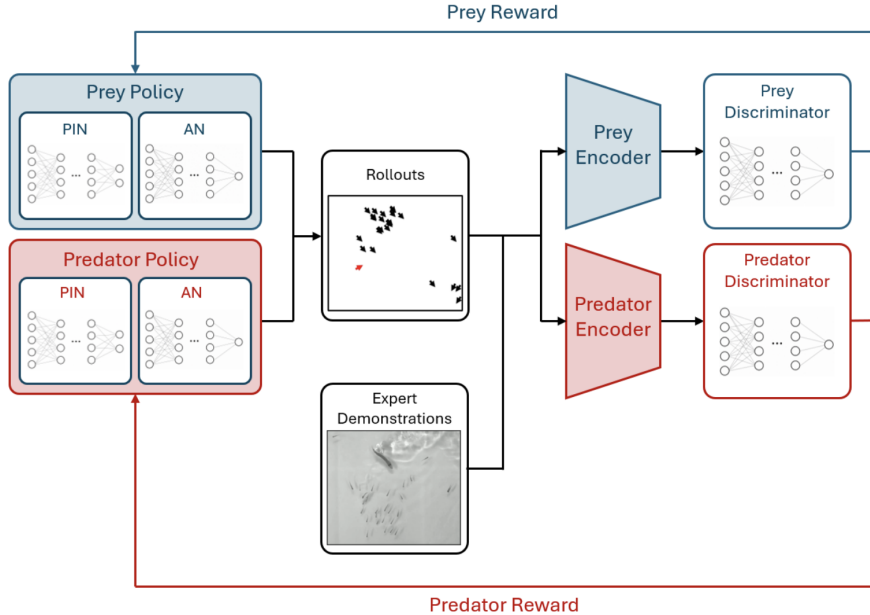


Figure 4.1: Overview of the GAIL architecture (Wirtheim & Kloss, in preparation).

4.1. Policy

The policy determines how an agent selects actions based on input features. Within the GAIL framework, the objective of the policy is to produce trajectories realistic enough so that the discriminator can no longer distinguish the policy’s trajectories from the expert recordings. Because the predator and prey pursue fundamentally different goals, such as the predator trying to capture a prey or the prey trying to escape, they each get their own policy instances. Thus, a dedicated policy governs the predator, and a second, shared policy governs every prey individual. The policy architecture used follows Wirtheim (2026), who was inspired by Heras et al. (2019) and Yapei Wu et al. (2024). It is built from two networks: the pairwise interaction network (PIN) and the attention network (AN) (Figure 4.2).

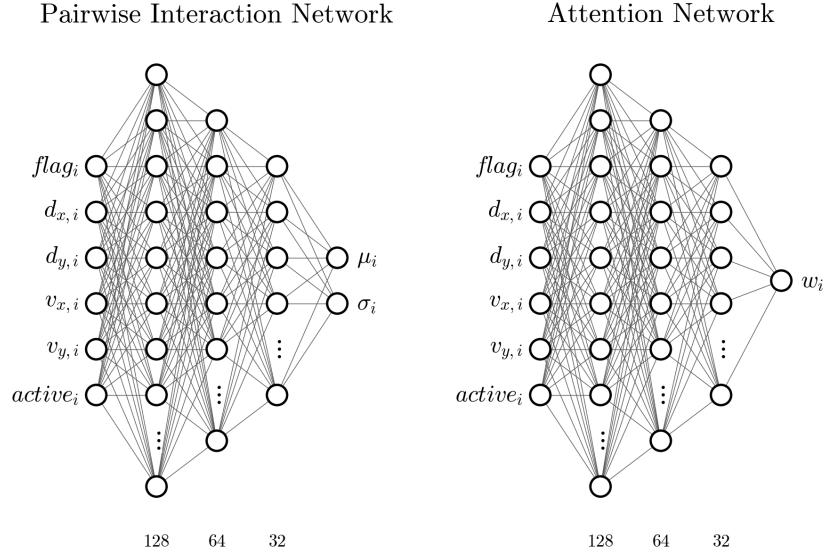


Figure 4.2: PIN and AN with the prey feature vector as input.

Separating interaction modeling (PIN) from aggregation (AN) keeps the structure interpretable and scalable to any number of neighbors. Both networks use three fully connected hidden layers with 128, 64, and 32 units and ReLU activations. The predator policy is written as $\pi_{P_{\phi,\varphi}}$, and the prey policy is written as $\pi_{Y_{\phi,\varphi}}$, with ϕ referring to the PIN parameters and φ to the AN parameters.

An alternative policy architecture was explored in which the PIN was replaced with a graph attention network (GAT). Because a GAT attends to all neighbors simultaneously, it was hypothesized that this might provide the policy with greater capacity to learn predator–prey interactions (Veličković et al., 2017). Distribution matching was at least as good as with the PIN and AN, but the directional escape metric did not improve, suggesting that policy capacity is not the factor limiting escape. The original architecture was therefore retained to maintain comparability with Wirtheim (2026).

4.1.1. Pairwise Interaction Network

The primary role of the PIN is to determine, for each neighbor separately, how the focal individual would react to that one neighbor. Specifically, the network learns how a neighbor’s relative distance and velocity influence the focal individual’s turning behavior. To model these interactions, the PIN receives a 5- or 6-dimensional feature vector for each neighbor. These features were introduced in Section 3.2.4 and include the neighbor’s relative position and velocity. The feature vector also contains a binary active-neighbor flag, marking whether the entry is a real fish or padding. This prevents the policy from treating padded, non-existent neighbors as real fish. For prey agents, the input includes an additional binary predator flag, enabling the policy to distinguish the predator from conspecific neighbors.

These features pass through three fully connected hidden layers, each more narrow than the last, with ReLU activations. Following Wirtheim (2026), these layers are deeper than the structure used by Yapei Wu et al. (2024) to capture the added complexity of behavior from real video data rather than simulation data, while remaining shallow enough for stable training. The output layer consists of two neurons that give the mean μ_i and standard deviation σ_i of a Gaussian distribution. The focal individual’s action toward a specific neighbor is sampled from the distribution:

$$a_i \sim \mathcal{N}(\mu_i, \sigma_i). \quad (4.1)$$

This stochasticity ensures that the policy does not always react the same way in the same situation. This resulting sample diversity helps mitigate compounding errors common to pure behavioral

cloning frameworks and reduces the risk of mode collapse. To ensure that the standard deviation remains positive and numerically stable, it is passed through a softplus transformation and offset by a small constant.

4.1.2. Attention Network

The primary role of the AN is to assign weights to each pairwise interaction modeled by the PIN. For each neighbor i , the network learns how strongly that neighbor should influence the focal individual’s action (Yapei Wu et al., 2024). Following Wirtheim (2026) and Yapei Wu et al. (2024), the AN has the same general architectural design as the PIN for both the predator and prey policies. The AN differs from the PIN only in its output layer. Rather than predicting a mean and standard deviation, the AN outputs a single raw attention score for each neighbor, which a softmax operation then turns into a normalized weight. For each neighbor $i \in \mathcal{I}$, the PIN produces an action a_i and the AN produces a corresponding weight w_i . The final action for the focal individual is the weighted sum over all neighbors:

$$a = \sum_{i \in \mathcal{I}} a_i \frac{w_i}{\sum_{j \in \mathcal{I}} w_j}. \quad (4.2)$$

Neighbors with larger attention weights have a greater influence on the final action, while neighbors with smaller weights contribute less. The resulting weighted sum is then passed through a sigmoid activation to transform the final action into the range $[0, 1]$.

Both the PIN and AN parameters are jointly optimized using evolutionary strategies (ES) (Salimans et al., 2017). This choice follows the approach of Wirtheim (2026) and Yapei Wu et al. (2024), who trained their GAIL policies using ES based on rank-normalized rewards and mirrored perturbations.

4.2. Transition Encoder

After generating rollouts from each policy, the resulting sequences must be compared against the expert tensors built in Chapter 3. The discriminator does this comparison by learning to distinguish between generated and expert behavior. However, processing complete sequences directly is problematic because it creates a high-dimensional discriminator input that grows quickly with the

number of agents, neighbors, frames, and features. Specifically, from a single prey-agent perspective in the 16-prey condition, the discriminator would process $1 \times 16 \times 6 = 96$ input values at once. For a full frame, this would grow to $17 \times 16 \times 6 = 1,632$ input values at once. Extending to 10-frame windows, the discriminator would need to process $17 \times 16 \times 6 \times 10 = 16,320$ input values at once, which is computationally expensive. While this is already a large number of inputs, the problem compounds with more agents. For the 32-prey condition, a 10-frame window would require processing $33 \times 32 \times 6 \times 10 = 63,360$ input values at once. This rapid growth highlights that using raw sequences as input to the discriminator does not scale well. A transition encoder addresses this issue by compressing the state sequences into a lower-dimensional representation before they reach the discriminator. This reduces input dimensionality while preserving the meaningful dynamics that actually matter for distinguishing expert behavior from generated behavior. Concretely, it learns a compressed representation of each state transition, so the discriminator receives a compact summary of how the state changes. The discriminator is then trained on these compact transition summaries rather than on the raw sequences directly. As with the policies, the predator and prey each get their own encoder. The predator’s encoder is written as $E_{P,\psi}$, and the prey’s is written as $E_{Y,\psi}$, each updating its own set of parameters ψ .

4.2.1. Encoder Design

Because the transition encoder works with the tensor representations, its architecture mirrors the tensor structure and state features detailed in Section 3.2.4. The design of the encoder follows Wirtheim (2026), whose approach was inspired by the masked video autoencoder from Yifan Wu et al. (2025). Yifan Wu et al. modeled collective behavior using latent representations of transitions between consecutive video frames. While Wirtheim kept this core idea of encoding transitions so that the discriminator can be trained on a compact representation of the original input data, he rebuilt the encoder’s internal structure to process expert trajectory tensors instead of raw video frames.

The transition encoder maps each agent’s neighbor and feature information into a compact transition representation that captures both its current state and how that state changes over time. It does this in three steps. The input to the encoder is a tensor of the shape established in Section 3.2.4:

$$[batch, frames, agents, neighbors, features].$$

First, at the neighbor level, the encoder aggregates information along the neighbor and feature dimensions for each agent. It uses the same attention-weighted method as the modular policy networks to produce a single embedding per agent:

$$[batch, frames, agents, embedding\ dimension].$$

Second, at an agent level, an MLP compresses each neighbor-aggregated embedding into a compact latent state z_t for each agent. This results in a per-agent latent state representation, which is the building block for the transition features:

$$[batch, frames, agents, z].$$

Third, at the transition level, the encoder computes how each agent’s latent state changes between two consecutive frames. For each time step t , it calculates:

$$\Delta z_t = z_{t+1} - z_t. \tag{4.3}$$

and concatenates this change with the current latent state z_t to form the transition representation:

$$e_t = [z_t, \Delta z_t]. \tag{4.4}$$

This preserves both the current latent state and its temporal change. Once the encoder is trained, this concatenation is the final tensor that eventually gets passed as input to the discriminator. It has the shape:

$$[batch, frames - 1, agents, 2z].$$

Since this comparison needs a pair of consecutive frames, a 10-frame window results in 9 transition representations per agent.

4.2.2. Encoder Training Using Self-Supervised Learning

Next, the encoders are trained to produce transition representations that capture the system’s meaningful dynamics and provide a stable input for the discriminator. This is done using self-supervised learning, which learns from unlabeled data directly (Gui et al., 2024). Following Wirtheim (2026), the encoders are pretrained separately on their corresponding expert trajectories using Variance-Invariance-Covariance Regularization (VICReg), which produces stable, well-structured embeddings. VICReg is a self-supervised method based on joint-embedding architectures. These methods create two augmented views of the same trajectory sequence and train the encoder to produce similar embeddings for both views, without using labels. A common problem with this type of architecture is representation collapse, where the encoder maps different inputs to nearly identical latent representations, resulting in embeddings with little structure. VICReg avoids this by adding two regularization terms to the embeddings (Bardes et al., 2021). Unlike autoencoders, which train the encoder to reconstruct the exact input, VICReg only requires the encoder to keep matching views close together in latent space.

Creating these two views relies on augmentations designed specifically for trajectory data. Three augmentations are applied. First, Gaussian noise is added to each neighbor’s relative position and velocity. This encourages the encoder to treat nearby positions as similar instead of memorizing exact coordinates. Second, neighbor dropout randomly removes an entire neighbor from the tensor, with each neighbor having a 10% chance of being masked. This reduces over reliance on any single neighbor. Third, feature dropout selects one feature and sets it to zero for every neighbor simultaneously, with a 5% probability. Because this removes information for every neighbor at once,

its rate is kept lower than the neighbor dropout rate. A few columns were excluded from all three perturbations. For the prey, the role-identifying binary flag is not touched. For both the predator and prey, the active binary column that marks real or padded neighbors is also protected. This prevents the padding signal from becoming unreliable during self-supervised pretraining.

Figure 4.3 shows the joint-embedding architecture used to compute the VICReg loss. It takes a batch of expert trajectories I as input. Each trajectory is augmented twice with different transformations, producing two views of the same sequence (X, X') . Both views pass through the encoder, which gives two latent-space representations, $Y = f_\theta(X)$ and $Y' = f_{\theta'}(X')$. A separate MLP, the projector h_ϕ , then maps these into an embedding space, $Z = h_\phi(Y)$ and $Z' = h_{\phi'}(Y')$, and the loss is computed on these embeddings.

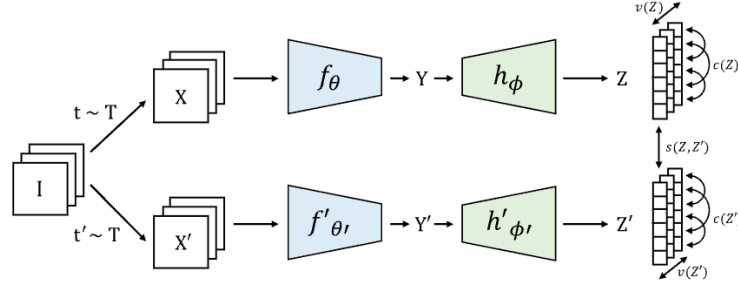


Figure 4.3: VICReg architectural overview (Bardes et al., 2021, p. 2).

The loss is made up of three components: invariance, variance, and covariance (Bardes et al., 2021). Invariance minimizes the mean squared distance between two embeddings. This pulls the two views closer, so small perturbations of the input do not change the encoder’s output much. Variance prevents representation collapse by enforcing a minimum standard deviation across embeddings of different trajectories. It has a hinge loss on the standard deviation to ensure that there is a minimum amount of variability for each embedding dimension. Covariance decorrelates the embedding’s dimensions, reducing redundancy so that no two dimensions end up encoding the same information.

The final loss combines these three terms as a weighted sum, and each encoder is pretrained separately on its own expert tensors from either predator or prey. Once pretraining ends, the projectors

are dropped and the encoder’s parameters are frozen. During GAIL, it only acts as a compressor so that expert and policy behavior are always compared in the same coordinates.

4.3. Discriminator

Now that the transition encoders have produced latent transition representations, separate role-specific discriminators, $D_{P,\beta}$ for the predator and $D_{Y,\beta}$ for the prey, are trained to differentiate between expert and generated behavior. Two discriminators are used because predator and prey have different behavioral patterns, leading to differing trajectories. For each agent, the discriminator receives the transition embedding $[z_t, \Delta z_t]$ as input and produces a scalar Wasserstein score. These scores are computed across all agents and transitions in the batch, resulting in a score matrix of shape $[\text{batch}, \text{frames} - 1, \text{agents}]$. Each discriminator functions like a critic, with higher scores meaning transitions are similar to expert behavior, and lower scores meaning otherwise. The discriminator is trained using a Wasserstein objective with a gradient penalty. The resulting score difference between expert and policy-generated transitions provides the role-specific feedback used to optimize the predator and prey policies.

4.3.1. Discriminator Design

The discriminator is a simple feedforward network consisting of three fully connected hidden layers with ReLU activations, followed by a final linear output layer that produces one scalar Wasserstein score for each agent-wise transition. What the discriminator receives as input consists of trajectory windows that are randomly sampled from either expert demonstrations or policy rollouts. The trajectory windows first pass through the frozen transition encoder, which produces a latent transition tensor of shape $[\text{batch}, \text{frames} - 1, \text{agents}, 2z]$. Specifically, for each agent at each time step, the encoder produces a compact representation by concatenating the latent state with its temporal difference $[z_t, \Delta z_t]$. Rather than operating on high-dimensional raw sequences, the discriminator evaluates these agent-wise transition features. It is this compact per-agent transition embedding, not the high-dimensional raw state, that gets flattened and fed into the discriminator’s MLP. Running the network once per agent per transition and stacking the resulting scalars back together produces the full output: a score matrix of shape $[\text{batch}, \text{frames} - 1, \text{agents}]$. To regularize the discriminator,

Gaussian noise is added to the state inputs during early training with linear decay over generations. This prevents the discriminator from growing overconfident too early and helps preserve the balance between the policy and discriminator updates. Both discriminators are optimized with RMSProp, following Wirtheim (2026), Yapei Wu et al. (2024), and Arjovsky et al. (2017), where it is the optimizer used for the Wasserstein critic. RMSProp keeps a running average of each parameter’s recent squared gradients and scales that parameter’s step by it. This means that a parameter that occasionally receives a large gradient cannot dominate the update (Tieleman & Hinton, 2012).

4.3.2. Wasserstein Loss with Gradient Penalty

The discriminator outputs the reward signal for policy updates by distinguishing the expert trajectories from the ones created by the current policy. Training the discriminator means improving its ability to differentiate the expert distribution $\tau_E \sim P_E$ and the policy distribution $\tau_\pi \sim P_\pi$, while the policy is trained to make the generated trajectories more expert-like (Arjovsky et al., 2017). As these distributions become more similar, the policy’s behavior more closely resembles the expert’s.

Standard GAIL uses binary cross-entropy loss, which minimizes the Jensen–Shannon divergence between the expert and policy distributions (Ho and Ermon, 2016). However, that divergence stops being informative when imitation learning begins with two distributions that barely overlap. Here, a discriminator would separate them almost perfectly, resulting in an uninformative learning signal. To avoid this, the Earth Mover (Wasserstein-1) distance is used instead, because it still reports how far apart two distributions are once they no longer overlap (Zhang et al., 2020). In the Wasserstein formulation, the discriminator is restricted to the class of 1-Lipschitz functions, and the distance is estimated as the difference in expected discriminator scores between expert and policy trajectories. To enforce the 1-Lipschitz constraint, a gradient penalty is used instead of weight clipping, following Wirtheim (2026), Gulrajani et al. (2017), and Yapei Wu et al. (2024). The penalty encourages the norm of the discriminator’s gradient with respect to its input to stay close to 1 on interpolations between expert and policy samples.

The discriminator is trained with the Wasserstein objective plus the gradient penalty term. The full discriminator loss is:

$$\mathcal{L}_D = \mathbb{E}_{\tau_E \sim P_E} [D_\beta(\tau_E)] - \mathbb{E}_{\tau_\pi \sim P_\pi} [D_\beta(\tau_\pi)] + \lambda \mathbb{E}_{\tau \sim \zeta} \left[(\|\nabla_\tau D_\beta(\tau)\| - 1)^2 \right]. \quad (4.5)$$

4.4. Environment

Every rollout begins from a frame taken directly from the expert demonstrations, so the group starts in a realistic configuration the fish actually produced. If the start were random, the first few steps would be spent moving away from an arrangement that never occurred in the data, yet the discriminator would score those steps anyway. Starting in a plausible position therefore improves training efficiency. Initial-state pools were built separately for the 16-prey and 32-prey conditions, providing a pool of starting positions and states for training. The learned policies are run in this environment to generate rollouts. These rollouts are produced in the same format as the expert data so that the discriminator can directly compare generated and expert trajectories. Each policy output $a \in [0, 1]$ is interpreted as a turn. It is rescaled to the interval set using the expert-derived maximum turn rate. The heading of the agent is updated, and the agent then advances one fixed step in the new direction. An agent that reaches a wall is placed back on the boundary with its heading mirrored, so the group stays inside the arena for the whole rollout.

4.5. Evaluation Metrics

Quantifying how closely the policy rollouts match the expert data distributions is essential for evaluating GAIL convergence. Each GAIL run is evaluated using the same six training metrics employed by Wirtheim (2026). This ensures comparability with his results and visualizations, especially given that he achieved convergence on prey-only data.

The first two metrics analyzed together are (1) the absolute discriminator scores for expert and policy-generated trajectories over generations and (2) the magnitude of the discriminator score difference over generations, used as an approximation of the Wasserstein distance. Since the main

goal is that the discriminator cannot differentiate between the expert and the policy, the Wasserstein proxy would ideally be close to zero. If the difference between the discriminator scores becomes large and the absolute scores of the expert and the policy drift apart, this would mean that the discriminator can easily separate the behaviors and identify the policy as fake.

The next two metrics examined together are (3) ES step size per generation and (4) the reward-difference standard deviation per generation. The ES step size gives insight into how strongly policy parameters change between consecutive generations. It is regularized by the update parameter α , which imposes smaller changes on each generation. The reward-difference standard deviation reflects how consistent the discriminator’s reward signal is across a generation. A higher value means a noisier and less reliable signal, while a lower value means a more consistent signal that gives clearer direction for the update. The objective is for the ES step size to stay within a narrow band rather than collapsing or diverging, and for the reward-difference standard deviation to narrow without reaching zero, which would mean the discriminator no longer separates the perturbed policies at all.

The final two metrics analyzed together are (5) Maximum Mean Discrepancy (MMD) and (6) Sinkhorn distance, both of which evaluate how the distributions align. The two metrics are used together to obtain robust and non-misleading results. A baseline for these metrics was established by repeatedly comparing two randomly sampled expert batches with a Monte Carlo (MC) approach, yielding a mean value and its variability. A policy is considered to be similar to the expert when the MMD or Sinkhorn distance reaches the expert baseline. Maximum Mean Discrepancy (MMD) is a kernel-based method that quantifies the similarity between two distributions by measuring the distance between their mean embeddings (Mehraban and Pichler, 2025). Sinkhorn comes from optimal transport theory (Luise et al., 2018) and measures how costly it is to move one distribution onto the other. Both metrics are computed on the transition embeddings, so they differ in how they compare the two samples. More weight is placed on Sinkhorn because it requires no kernel-bandwidth choice, whereas the MMD value depends on the bandwidth heuristic and is therefore harder to compare across runs.

4.6. Training Procedure

The following is the GAIL algorithm training loop.

Algorithm 1 GAIL procedure to imitate predator-prey dynamics

Input: Policies $\pi_{P_{\phi,\varphi}}$ and $\pi_{Y_{\phi,\varphi}}$, Discriminators $D_{P,\beta}$ and $D_{Y,\beta}$,
Encoders $E_{P,\psi}$ and $E_{Y,\psi}$, Expert trajectories $\tau_{E,P}$ and $\tau_{E,Y}$
Output: Best predator $\pi_{P_{\phi,\varphi}}^*$ and prey policy $\pi_{Y_{\phi,\varphi}}^*$

- 1: Pretrain $\pi_{P_{\phi,\varphi}}$ and $\pi_{Y_{\phi,\varphi}}$ with BC
- 2: Train and freeze $E_{P,\psi}$ and $E_{Y,\psi}$
- 3: Initialize EMA policies $\bar{\pi}_{P_{\phi,\varphi}}$ and $\bar{\pi}_{Y_{\phi,\varphi}}$
- 4: **for** $i = 0, 1, 2, \dots, M$ generations **do**
- 5: Sample group-size condition $n_{\text{prey}} \in \{16, 32\}$
- 6: Execute $\bar{\pi}_{P_{\phi,\varphi}}$ and $\bar{\pi}_{Y_{\phi,\varphi}}$ to collect rollouts under n_{prey}
- 7: **for** $j = 1, \dots, J$ $D_{P,\beta}$ -update-ratio **do**
- 8: Update predator discriminator $D_{P,\beta}$
- 9: **end for**
- 10: **for** $j = 1, \dots, J$ $D_{Y,\beta}$ -update-ratio **do**
- 11: Update prey discriminator $D_{Y,\beta}$
- 12: **end for**
- 13: Optimize predator PIN $\pi_{P_{\phi}}$ and AN $\pi_{P_{\varphi}}$ with ES
- 14: Optimize prey PIN $\pi_{Y_{\phi}}$ and AN $\pi_{Y_{\varphi}}$ with ES
- 15: Update EMA: $\bar{\pi}_{P_{\phi,\varphi}} \leftarrow \text{EMA}(\pi_{P_{\phi,\varphi}})$, $\bar{\pi}_{Y_{\phi,\varphi}} \leftarrow \text{EMA}(\pi_{Y_{\phi,\varphi}})$
- 16: Apply decay γ on learning rates λ_P, λ_Y and exploration factor σ_P, σ_Y
- 17: **end for**

Hyperparameter settings were set before training. Wirtheim (2026) initially adopted hyperparameter values from Yapei Wu et al. (2024) and fine-tuned until training stabilized. In this study, the final hyperparameters from Wirtheim (2026) were used as the starting configuration, providing a well-tuned baseline.

The training configurations consisted of:

- **num_generations:** number of training generations.
- **gamma:** learning-rate decay factor.
- **performance_eval:** number of generations between performance evaluations.
- **num_perturbations:** number of perturbations for ES evaluated per submodule and generation.

For the prey, the setup included:

- `lr_prey_policy`: learning rate for the prey policy, controlling update step sizes.
- `sigma_prey`: prey ES exploration noise, controlling perturbation magnitude.
- `prey_dis_balance_factor`: number of prey discriminator updates per generation.
- `prey_noise`: prey discriminator input noise.
- `lr_prey_disc`: learning rate for the prey discriminator, controlling update step sizes.
- `lambda_gp_prey`: prey gradient-penalty coefficient.
- `prey_update_mode`: method used to compute the discriminator-based reward, including additional avoidance rewards.

For the predator, the setup included:

- `lr_pred_policy`: learning rate for the predator policy, controlling update step sizes.
- `sigma_pred`: predator ES exploration noise, controlling perturbation magnitude.
- `pred_dis_balance_factor`: number of predator discriminator updates per generation.
- `pred_noise`: predator discriminator input noise.
- `lr_pred_disc`: learning rate for the predator discriminator, controlling update step sizes.
- `lambda_gp_pred`: predator gradient-penalty coefficient.
- `predator_update_mode`: method used to compute the discriminator-based reward, including additional attack rewards.

The list of software used in training can be found in the Appendix.

4.6.1. Joint Discriminator for Fine-Tuning

As an additional exploration, a joint discriminator was tested as a fine-tuning mechanism on top of the separate predator and prey discriminators. Training was organized into two stages. Stage 1, role-specific training, follows the procedure described above, with separate policies, transition encoders, and discriminators for predator and prey. The best-performing policies from this stage were saved. Stage 2, joint fine-tuning, uses these saved policies as its starting point. They are

further fine-tuned using a single joint discriminator, with the aim of capturing more predator-prey interactions.

Creating a joint discriminator was inspired by Wirtheim (2026), who identified the decoupled discriminators as a possible cause of the missing inter-group dynamics. The joint discriminator follows the same architecture as the role-specific discriminators. To handle multiple agents, mean pooling over the agent dimension is applied for each role, with the prey group and the predator each being reduced to a single fixed-size vector per time step. Whereas the role-specific discriminators output a score per agent, with shape [batch, frames, agents], the joint discriminator outputs a single score per frame for the entire system, with shape [batch, frames]. This allows the discriminator to evaluate the collective state of the predator-prey system.

4.6.2. Training Stabilization

Training a policy and discriminator within GAIL can lead to instability. The reward signal is at risk of degrading if the discriminator learns too quickly, meaning the policies cannot adapt. Since two policies are trained at once, these problems are amplified because they both change the environment in which they are learning. GAIL could face mode collapse, meaning the generator only produces a certain behavior type that the discriminator favors (Ho and Ermon, 2016; Yifan Wu et al., 2025). Following Wirtheim (2026), several measures were taken to keep both sides of the training balanced and to mitigate some of these potential issues.

On the discriminator side, the number of discriminator updates per policy update was set separately for each role, so that the critic can stay informative without the policy being overwhelmed. Additionally, Gaussian noise was added to the discriminator’s state inputs and decayed linearly over the first half of training, which prevents the discriminator from becoming overconfident before the policies have had a chance to improve. On the policy side, both policies are initialized through behavioral cloning on the expert actions, so that evolutionary strategies start from a policy that already has an understanding of the expert action distribution, rather than a policy that starts by reproducing random behavior (Ho and Ermon, 2016). Each ES update is then clipped to a fixed fraction of a reference parameter norm recorded at initialization (Liu et al., 2019). This means

that no single generation can move the policy far enough to invalidate the discriminator’s feedback. Rollouts are generated from an exponential moving average (EMA) of the policy weights rather than the current ones, so that the behavior the discriminator sees is smoothed across generations (Morales-Brotons et al., 2024). Both learning rates and exploration noise are decayed geometrically each generation, so that exploration is broad early on and updates become progressively smaller as training proceeds (Yapei Wu et al., 2024).

A few measures handle the training signal itself. Since rollouts and expert windows are padded to a common shape, the rewards, discriminator scores, and behavioral cloning targets are averaged over the active agents only, so that non-existing fish are not accounted for. This means that the signal reflects real behavior rather than any of the padded entries. Relative velocities are clipped and normalized by a fixed maximum speed derived from the expert speed distribution. This keeps the expert and generated inputs on the same scale. Evaluation metrics are averaged over several independently generated episodes, and the policies that are kept are the best-scoring EMA weights instead of the final ones. Lastly, all runs use a fixed random seed to make them reproducible.

Training Results

Wirtheim (2026) previously demonstrated that GAIL can be successfully trained when applied to a single role in isolation, achieving convergence for a prey-only policy against Couzin-simulated data. This thesis attempts to extend the successful homogeneous baseline to the full predator-prey setting. Building on the co-evolutionary approach of Yapei Wu et al. (2024), the same underlying GAIL and evolution-strategies implementation is adapted here into a framework in which the predator and prey policies are optimized as two separate, modular subproblems. Training progress is monitored using the six metrics introduced in Section 4.5: (1) absolute scores, (2) score differences, (3) ES step size, (4) standard deviation of reward differences, (5) MMD, and (6) Sinkhorn.

The co-evolutionary framework was verified under increasingly complex conditions, with each stage also testing the extensions introduced in Chapter 3. First, behavioral cloning was used to establish whether the padded neighbor dimension with its explicit active mask, which allows the 16 and 32 prey trials to be trained within one model, could be learned at all and to provide a strong comparison baseline. Second, the GAIL training procedure was evaluated on synthetic predator-prey demonstrations with predefined interaction dynamics. Third, after this synthetic verification, the same modeling framework was transferred to trajectories reconstructed from real predator-prey recordings, where the hand-labeled 16-prey dataset built in this thesis raises the available training data from 793 windows of the previous work to 3,771 windows. Fourth, training was restricted to successful attacks alone, isolating the inter-group dynamics that motivate this thesis. Finally, whereas the previous steps used separate discriminators for predator and prey, an additional joint discriminator was introduced as a fine-tuning stage on top of the separately trained models, allowing the reward signal to depend on the interaction between both roles rather than each in isolation.

5.1. Behavioral Cloning

Despite the shortcomings of behavioral cloning mentioned in Section 2.3.1, it remains a foundational approach to the imitation learning problem because of its simplicity and its ability to learn a policy directly from expert demonstrations via supervised learning (Torabi et al., 2018). In this work, behavioral cloning serves several purposes. First, it verifies that the input representation, which uses padded neighbor slots to handle varying prey sizes, can be learnt. Second, it provides a strong supervised baseline for comparison with the GAIL results. Finally, behavioral cloning is used to pretrain the policy network on expert trajectories from real video data before applying GAIL, allowing training to start from a policy that already behaves similarly to the expert. This stabilizes subsequent training without requiring any environmental interaction (Ho and Ermon, 2016).

For the behavioral cloning procedure, the expert trajectories are flattened into state-action pairs, and samples in which an agent had no valid (non-padded) neighbors are removed by a presence-flag threshold, so that only samples with meaningful neighborhood information are kept. The filtered dataset is then split into 80% training and 20% validation data. The policy is optimized with Adam ($\text{lr} = 1\text{e-}3$) to minimize MSE between predicted and expert actions, using deterministic forward passes for gradient stability. Training runs for up to 250 epochs with early stopping based on the validation loss, preventing the policy from overfitting to the training trajectories and ensuring it generalizes within the expert’s state distribution. The best validation checkpoint is saved and later used to initialize the corresponding policy network for GAIL training with real video data.

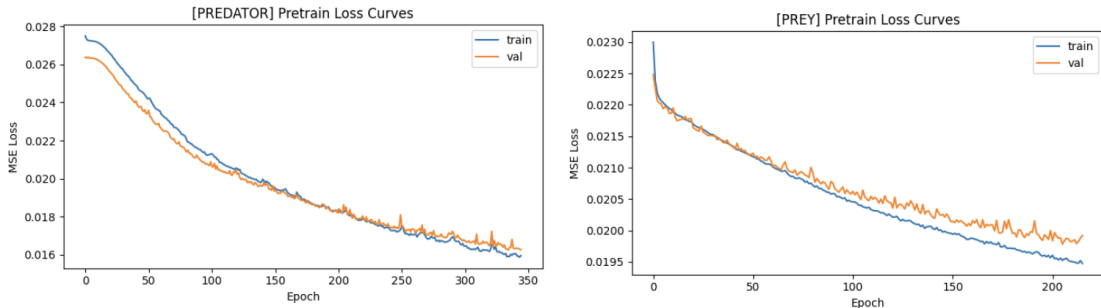


Figure 5.1: Behavioral cloning pre-training loss curves for predator and prey.

Both the predator and prey policies show a smooth MSE decay in training and validation loss, confirming that the padded neighbor input representation is learnable. For the predator policy, early stopping occurred at epoch 345 and for the prey policy, early stopping occurred earlier at 216, consistent with the prey's validation loss becoming slightly noisier in its later epochs (Figure 5.1).

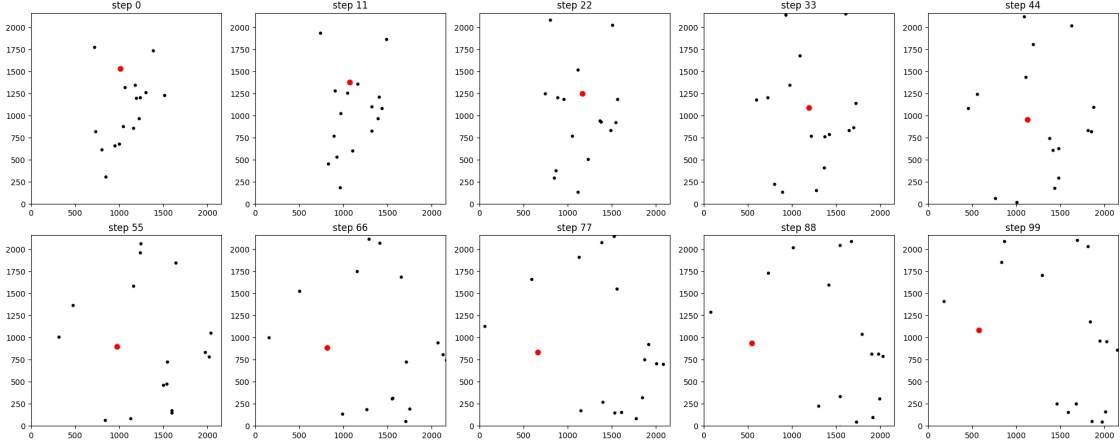


Figure 5.2: Behavioral cloning rollouts with 16 prey.

From these results, it is evident that behavioral cloning produces plausible per-step actions and that padded-neighbor representation is learnable. However, the rollouts in Figure 5.2 demonstrate that the resulting policies do not show directed pursuit or evasion, motivating the progression to GAIL as well as the attack-focused analysis presented in Section 5.4, which restricts training to windows containing only successful attacks.

5.2. Predator-Prey Policy Learning from Synthetic Data

The second stage of the analysis applies GAIL to predator-prey trajectories generated by a Couzin-style simulation, following Yapei Wu et al. (2024), who similarly trained models on Couzin-generated data. This provides a fully controlled setting in which to verify the framework’s ability to recover predator-prey behavior from synthetic demonstrations, before introducing noisy real data.

The synthetic expert demonstrations were generated using an adapted version of the predator-prey simulation provided by Haeri and implemented by Wirtheim (2026), which is an extension of

the classic Couzin model (Couzin, 2002) with an added predator. Prey motion follows the local repulsion, orientation, and attraction rules of the Couzin model. Each simulation was conducted within a 50x50 two-dimensional arena for 500 steps, with agents moving at a fixed speed of 5 units per unit time, a step size of 0.5, and a maximum per-step turning angle of 0.25.

The predator follows a fixed, deterministic pursuit rule that targets whichever prey individual is nearest and currently visible (I is the set of prey visible to the predator):

$$i_t^* = \arg \min_{i \in I} \|\mathbf{x}_{i,t} - \mathbf{x}_{P,t}\|, \quad (5.1)$$

The corresponding desired pursuit direction is given by:

$$\mathbf{d}_{P,t} = \frac{\mathbf{x}_{i_t^*,t} - \mathbf{x}_{P,t}}{\|\mathbf{x}_{i_t^*,t} - \mathbf{x}_{P,t}\|}. \quad (5.2)$$

Similar to the prey, the predator is constrained by the same maximum per-step turning angle and advances at the same constant speed each timestep. The prey continue to follow the Couzin flocking rules with an added avoidance response triggered whenever the predator enters a fixed threat radius around an individual. The resulting trajectories are clean demonstrations with no missing detections. They have known interaction dynamics that avoid uncertainties from noisy biological tracking.

To isolate the performance of GAIL, the policies in this stage were not pretrained with behavioral cloning. Further, GAIL training with Couzin-generated data follows a curriculum learning strategy (Bengio et al., 2009), in which either the 16-prey or 32-prey condition is sampled each generation. Rather than sampling both conditions with equal probability from the start, which was found to destabilize training, the probability of selecting the 16-prey condition increases linearly from 0.2 to 0.5 over the first 30% of training generations and is then held at an even split for the remainder. This allows the policies to stabilize first with the 32-prey condition before the gradual introduction of the 16-prey condition in equal measure, avoiding the instability seen when both conditions were

sampled evenly. The corresponding hyperparameter configuration is reported in Figure A.2 in the Appendix.

Before starting GAIL training, the prey and predator transition encoders are pretrained on the expert Couzin data using a VICReg objective. Figure 5.3 gives a closer look at the loss components. For the prey encoder, invariance drops quickly early in training and stays low throughout, drifting gradually upward in later epochs. Variance declines steadily over training indicating that the encoder maintains diverse embeddings instead of collapsing to a point. Covariance rises before stabilizing, indicating residual correlations between embedding dimensions. For the predator encoder, invariance follows a similar pattern to the prey, but with a less pronounced upward trend. Variance only moderately decreases after the initial drop, pointing to a less thorough use of the embedding space. Covariance increases and stabilizes as well but with noticeably more fluctuation. Both encoders are judged adequate as discriminator feature extractors.

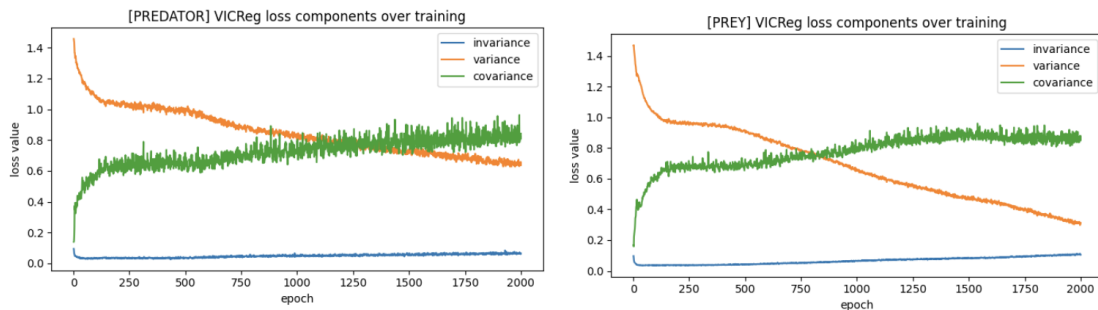


Figure 5.3: VICReg pretraining losses for the Couzin predator-prey transition encoders.

Turning to the discriminator (Figure 5.4), the prey score difference peaks between generations 1200 and 1700, then drops sharply by generation 2000, before slightly drifting up again to a low level by the end of training. The predator’s score difference is more volatile throughout the generations and ends in a similar pattern to how it began. In both cases, the gap does not grow uncontrollably and remains bounded, which is the stability required before moving to real data.

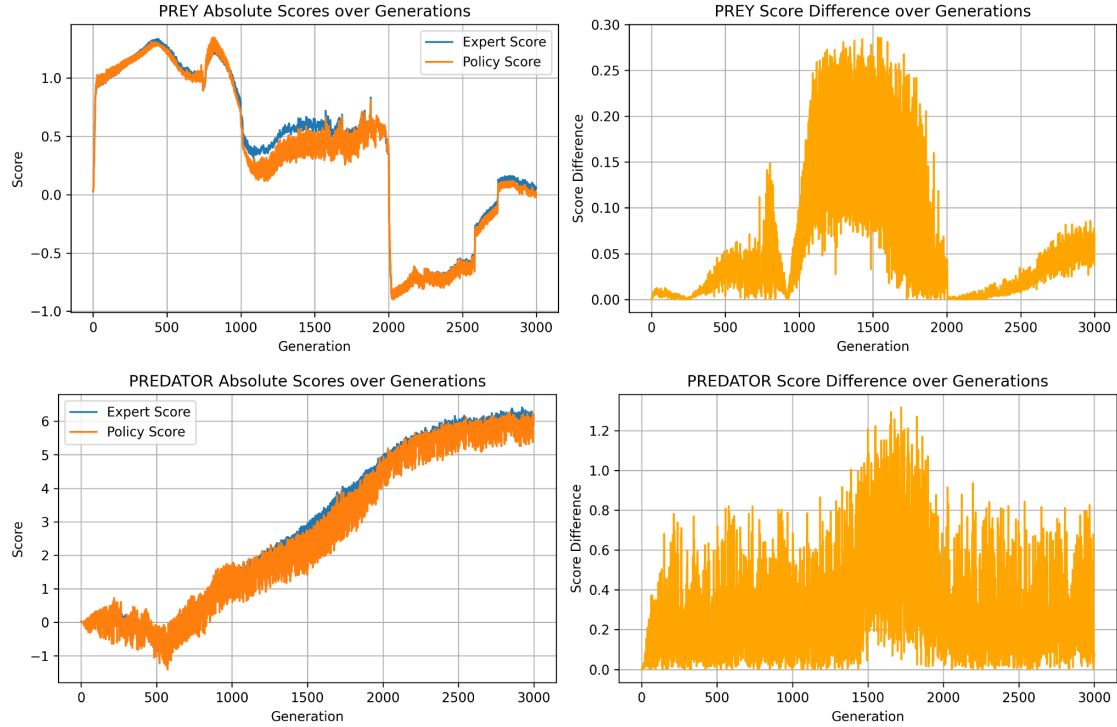


Figure 5.4: Discriminator scores and score difference over generations for the Couzin predator-prey model.

As seen in Figure 5.5, for both the predator and the prey, the ES update magnitude stays within a narrow band for both roles, so the updates neither die out nor blow up. The reward-difference standard deviation, which is how much gradient signal the evolution strategy receives, declines for the PIN and remains low and approximately flat for the AN, with the two converging to a similar level after about generation 1500 in both roles. The signal narrows without collapsing, remaining above the threshold where a generation is considered as dead, so a usable ES gradient is available throughout training.

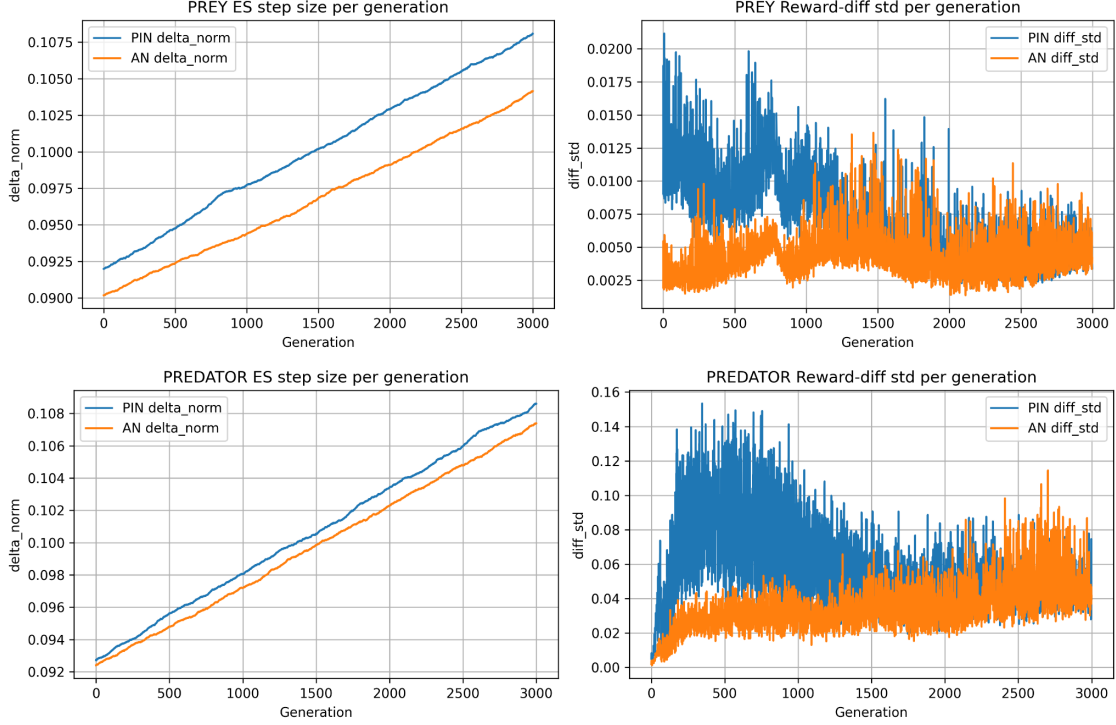


Figure 5.5: ES update step size and reward difference standard deviation over generations for the Couzin predator-prey model.

For the prey, both Sinkhorn and MMD spike when the curriculum strategy finishes ramping up (Figure 5.6). After this spike, at around generation 1000, Sinkhorn settles and remains stable near the expert baseline, while MMD remains above the baseline. For the predator, Sinkhorn does remain consistently steady and closer to the baseline. MMD, on the other hand, is a bit noisier and remains above the baseline, but does not show an upward trend.

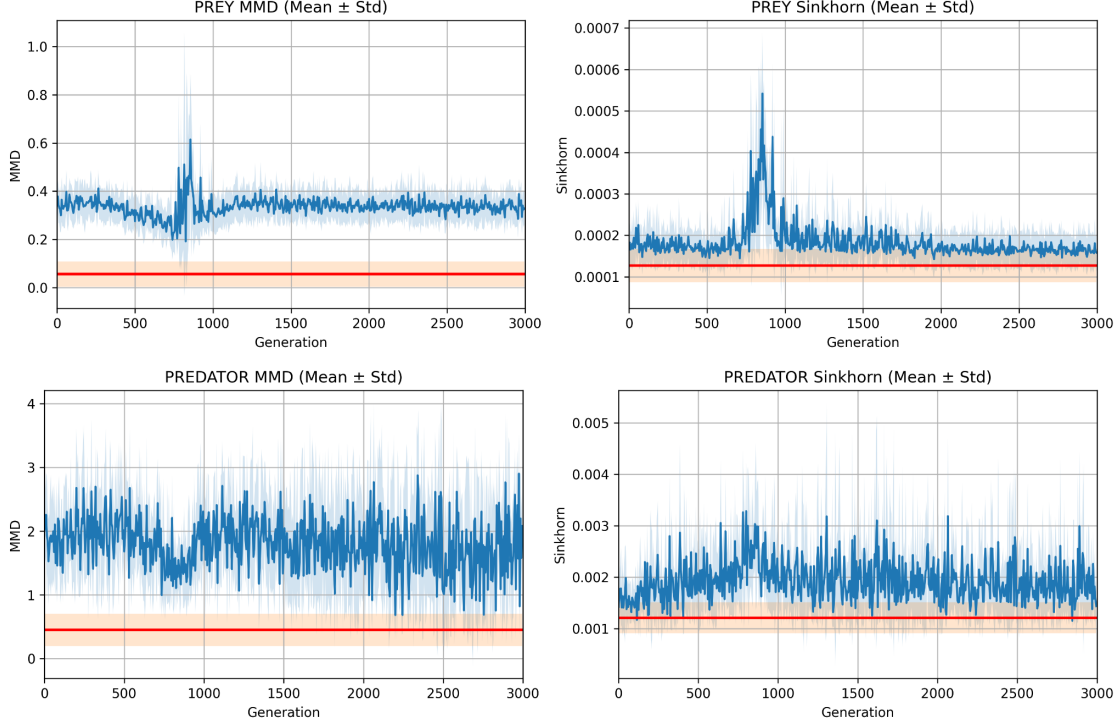


Figure 5.6: MMD and Sinkhorn distances over generations for the Couzin predator-prey model.

All the training metrics evaluated together show that the co-evolutionary framework trains stably on Couzin data with both population sizes.

5.3. Predator-Prey Policy Learning from Biological Data

After verifying the framework on synthetic predator-prey interactions, the same architecture is applied to trajectories reconstructed from biological recordings. Unlike the Couzin verification stage, this stage starts with behavioral cloning pretraining, and instead of following a curriculum strategy, the 16 and 32 prey conditions are sampled with equal probability throughout training. This sampling strategy was found to be more stable than the curriculum approach in combination with BC training. The corresponding hyperparameter configuration is reported in Figure A.3 in the Appendix.

As in the previous stage, the prey and predator encoders are pretrained on real video data for 4000 epochs before beginning GAIL training. For the prey encoder, invariance sits at a low value from

early in pretraining and slightly increases over the remaining epochs. Variance declines consistently, dropping below the invariance term and approaching zero, indicating that the prey embedding dimensions remain fully diverse. Covariance initially increases, then begins to decrease slightly around epoch 1500, showing more fluctuation than the other two components. For the predator encoder, invariance follows a similar pattern to the prey. Variance steadily declines during training, indicating that the encoder utilizes the entire embedding space throughout training. Covariance initially rises before stabilizing at a noisier plateau, implying residual correlations (Figure 5.7). Both encoders are considered suitable as feature extractors for the discriminator.

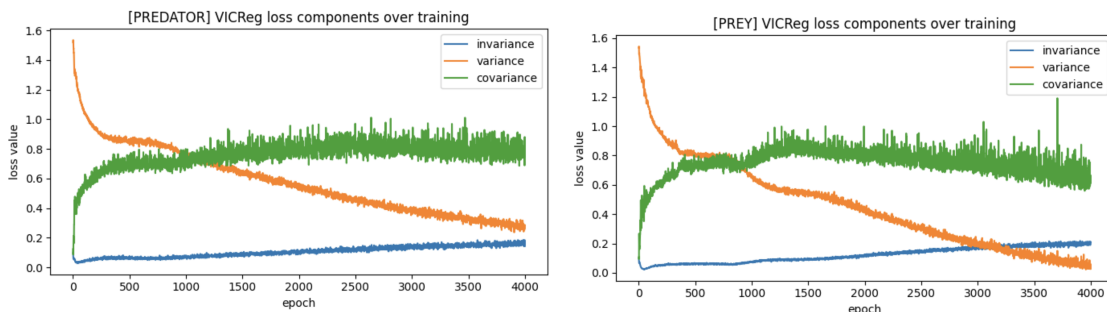


Figure 5.7: VICReg pretraining losses for the video predator-prey transition encoders

In Figure 5.8, the prey's score difference grows steadily over the first thousand generations, then remains there for the rest of training. The predator's absolute score drops sharply towards zero around generation 750, which corresponds with the score difference narrowing and remaining lower, albeit with some fluctuations. This suggests that policy and expert trajectories become more difficult to distinguish during this period. Though this convergence weakens again from generation 2000 onwards, with the score difference becoming noisier.

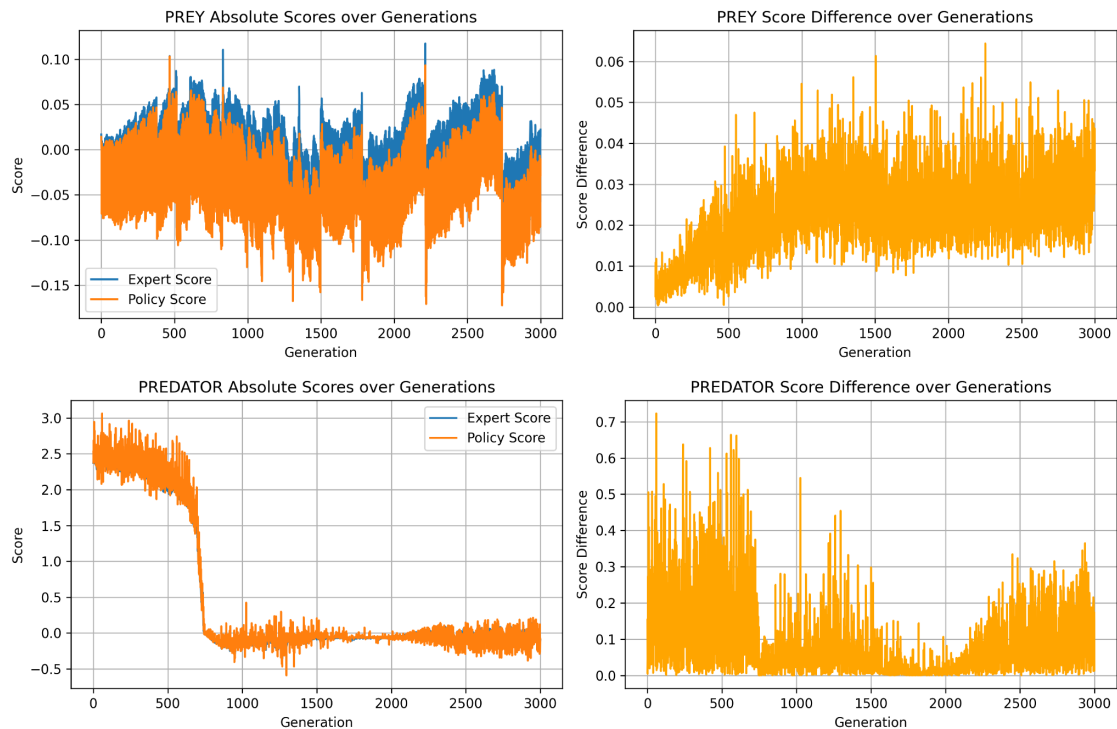


Figure 5.8: Discriminator scores and score difference over generations for the video predator-prey model.

For both the predator and the prey, ES step size grows throughout training, with a clear change of slope at around generation 800 for the predator and at a slightly later stage for the prey at generation 1250 (Figure 5.9). Examining the reward-difference standard deviation, for the prey, it peaks around generation 500, after which it gradually declines and settles into a noisy trend for the remainder of the training period. For the predator, the reward-difference standard deviation is elevated and noisy until around generation 750, when there is a sharp drop. It then remains low until generation 2000, when it begins to rise again. This mirrors the widening seen in the above score-difference plot.

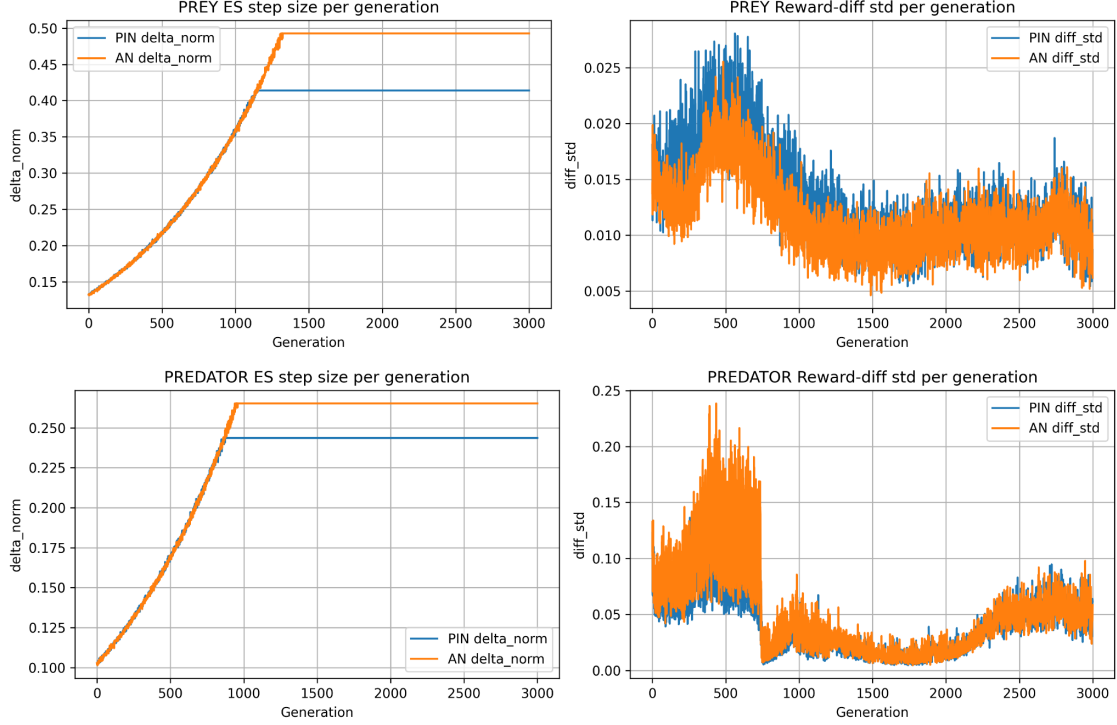


Figure 5.9: ES update step size and reward difference standard deviation over generations for the video predator-prey model.

As seen in Figure 5.10, for the prey, both Sinkhorn and MMD spike around generation 500 before settling and stabilizing around generation 1000. Sinkhorn settles near the top of the expert's baseline range, whereas MMD remains above the baseline throughout training. Similarly, Sinkhorn for the predator also settles close to the baseline after an early spike, while MMD remains above the baseline and is highly variable, though does gradually drift downward throughout training.

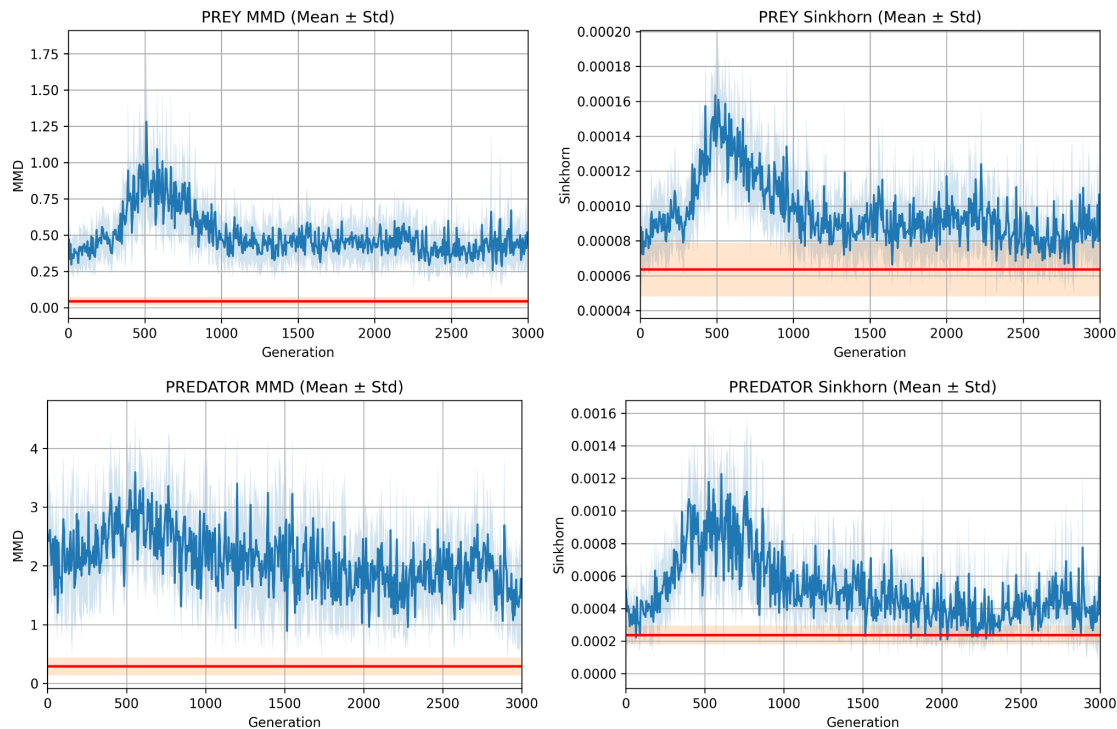


Figure 5.10: MMD and Sinkhorn distances over generations for the video predator-prey model.

While the metrics used to evaluate training performance suggest meaningful learning, aggregate distribution distances do not capture whether the interaction itself is reproduced, so the rollouts must also be examined directly. The rollout (Figure 5.11) shows that the learned policies can imitate expert behavior, with a predator successfully pursuing a prey, but the prey group fails to maintain tight cohesion. Given this, the next stage restricts training to successful attack interactions, testing whether a more curated subset of real predator-prey behavior yields better results.

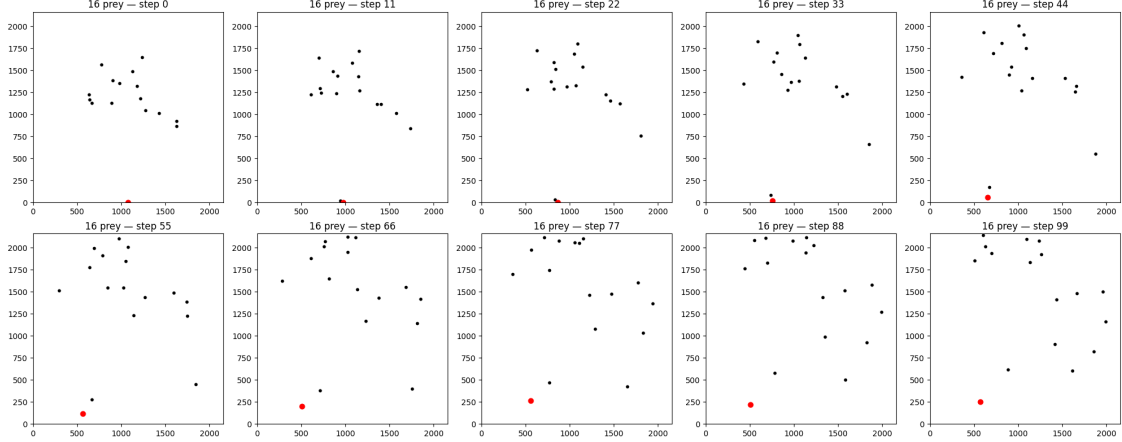


Figure 5.11: GAIL rollouts with 16 prey.

5.4. Predator-Prey Policy Learning from Only Successful Attacks

While the preceding stages were trained on both meaningful interactions and successful attacks, this stage tests whether this data composition, rather than model capacity, limits the recovery of predator pursuit and cohesive prey escape. Although the interaction clips were already curated to contain behaviorally relevant episodes such as failed attacks, active pursuit, and evasion, and any instance where predator movement causes the prey to move differently than their typical schooling, the last criterion admits a broader range of behavior. This stage therefore restricts training to the hand-labeled attack clips alone, across both the 16- and 32-prey conditions, so that every training window contains a successful attack. The resulting subset contains 2,200 windows, which is less than the 3,771 used in Section 5.3, but now dense in the behavior of interest.

As a first step, a behavioral cloning policy was pretrained on this attack-only subset of the video data, following the same procedure described in Section 5.1. While the previous behavioral cloning policy that contained all predator-prey interactions did not exhibit sustained pursuit or evasion (Figure 5.11), the attack-trained policy shows more consistent tracking behavior with the predator following the prey group throughout the rollout (Figure 5.12).

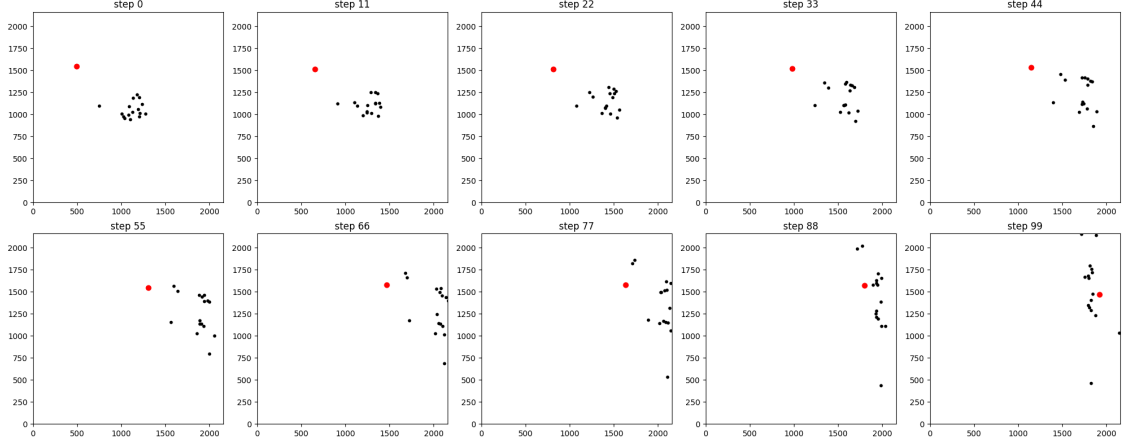


Figure 5.12: Behavioral cloning rollouts with 16 prey, focusing only on attack data.

GAIL was therefore applied to the successful attacks alone, to establish whether the training metrics converge more closely under a curated interaction distribution. The architecture and hyperparameter configuration are identical to Section 5.3 with only the data differing. The prey and predator transition encoders were pretrained again, this time on the attack data alone.

The resulting VICReg loss components are similar to those of the previous stage (Figure 5.13). For the prey encoder, the invariance term remains low and increases slowly over training. The variance term drops below the invariance term, and does so earlier than in the previous stage, suggesting that the prey embedding reaches diversity sooner when trained on the narrower attack distribution. The covariance term initially increases and, as in the previous stage, begins to steadily decrease from around epoch 1500. For the predator encoder, the invariance term follows a similar pattern to the prey, but the variance term does not fall below it. The covariance term rises before plateauing rather than entering a downward trend.

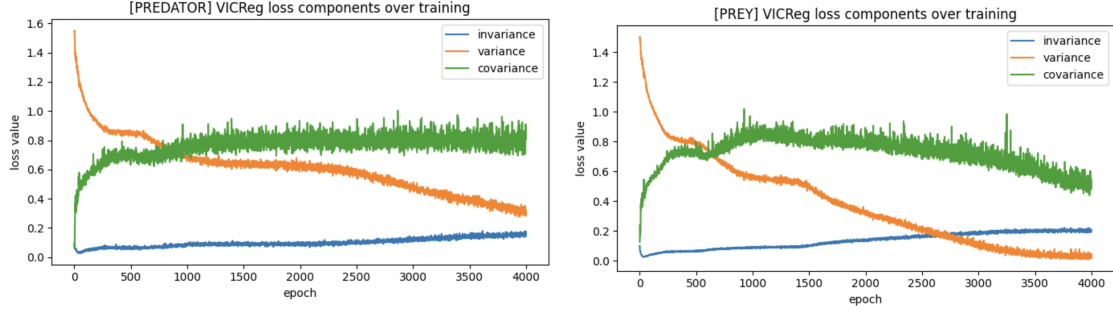


Figure 5.13: VICReg pretraining losses for the video predator-prey transition encoders, focusing only on attack data.

The discriminator scores fluctuate throughout training for both roles. For the prey, the expert and policy absolute scores rise together with a steeper climb from generations 750 to 1500. The score difference falls close to zero around generation 500, and then gradually grows with fluctuations. For the predator, both scores fall sharply around generation 500, then recover steadily until generation 1750 when it starts to slowly decline again. The predator score difference is noisier, with a dip closer to noise around generation 1500, but then more active noise peaking around generation 2000. In both roles the gap remains bounded and does not diverge (Figure 5.14).

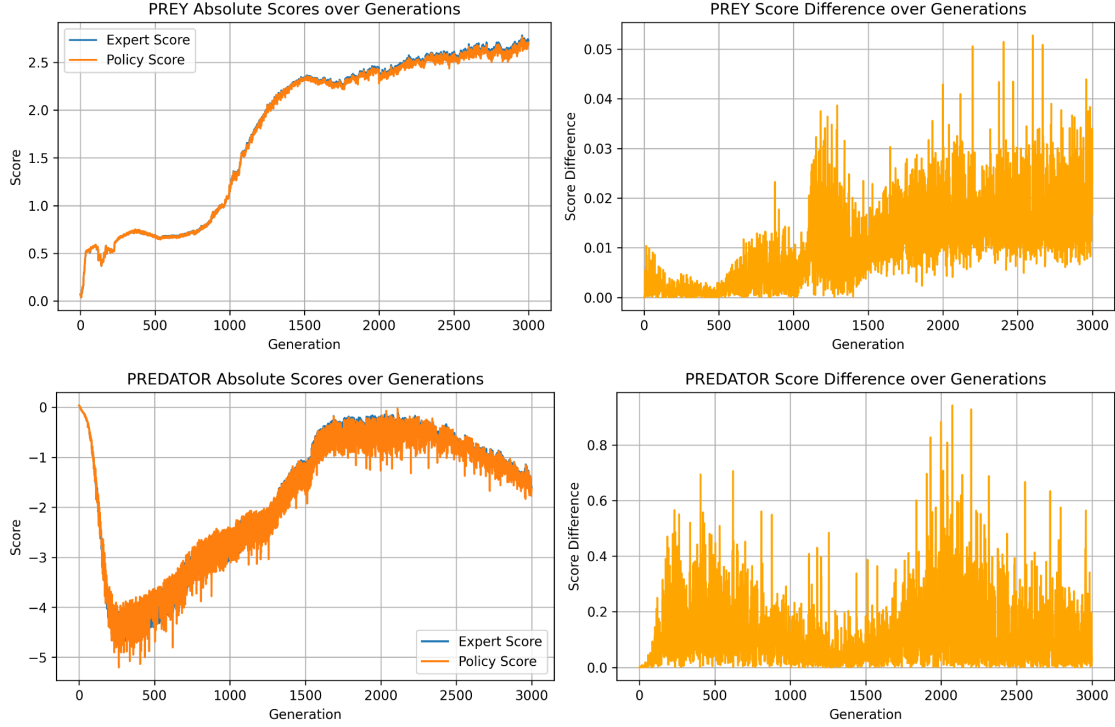


Figure 5.14: Discriminator scores and score difference over generations for the video predator-prey model, focusing only on attack data.

As seen in Figure 5.15, the ES step size grows until around generation 800 for the predator and generation 1050 for the prey, and then becomes constant. The reward-difference standard deviation for the prey has three stages of decline. Generation 0 to 500 there is decline then an increase, then from generations 750 to 1750 another decline and another jump, and finally from generation 1750 until 3000 there is a steady decline. The predator has from generation 250 to 1750 a decline and also then an increase, and it starts declining again from generation 2000 to 3000. In both roles the signal narrows without collapsing, meaning a usable ES gradient persists throughout. Also, for both roles the curves are indistinguishable after around generation 1000 meaning that the critic does not favor one component of the policy over the other.

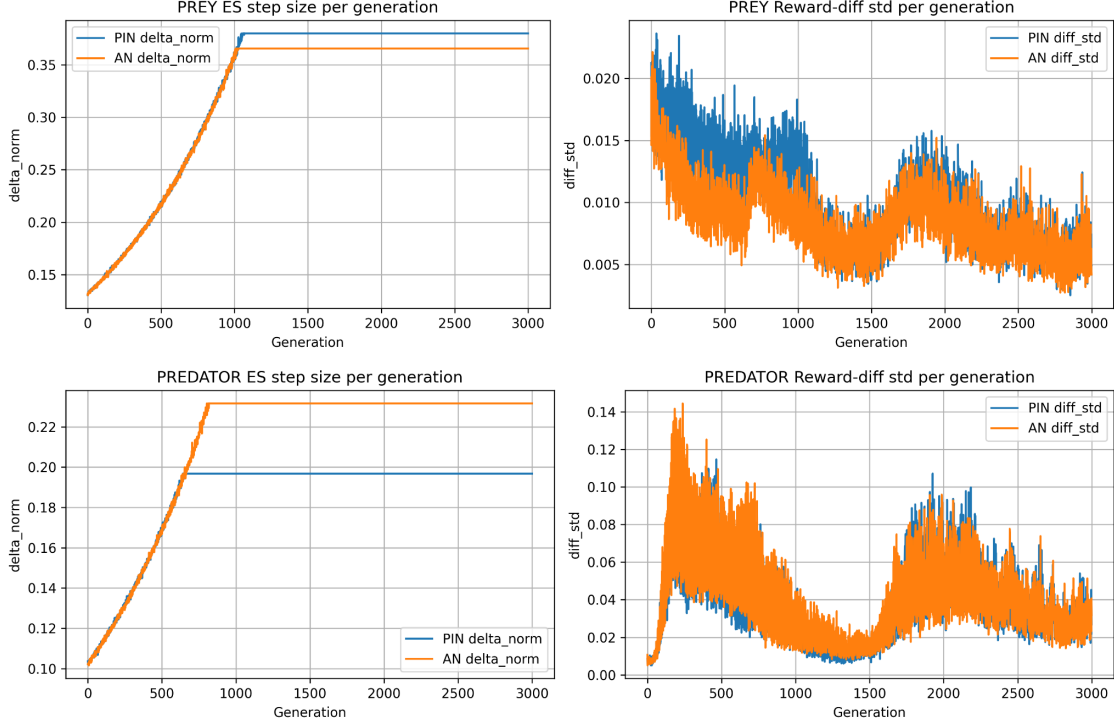


Figure 5.15: ES update step size and reward difference standard deviation over generations for the video predator-prey model, focusing only on attack data.

For the prey, the MMD has a spike around generations 750 to 1000, then plateaus, then has a smaller spike around 1800, where it then once again plateaus slightly above the expert baseline. For the Sinkhorn, it follows the same pattern as the MMD but converges far more closely (Figure 5.16). The attack-trained prey policy therefore reaches the expert baseline under the Sinkhorn distance and holds it for the final third of training. This is a level of distribution matching that neither the Couzin nor the previous stage using all data managed to attain. For the predator, MMD fluctuates with a slight decline but remaining more elevated above the baseline. The Sinkhorn does decrease and approach the expert baseline but with more noise than the clear prey convergence.

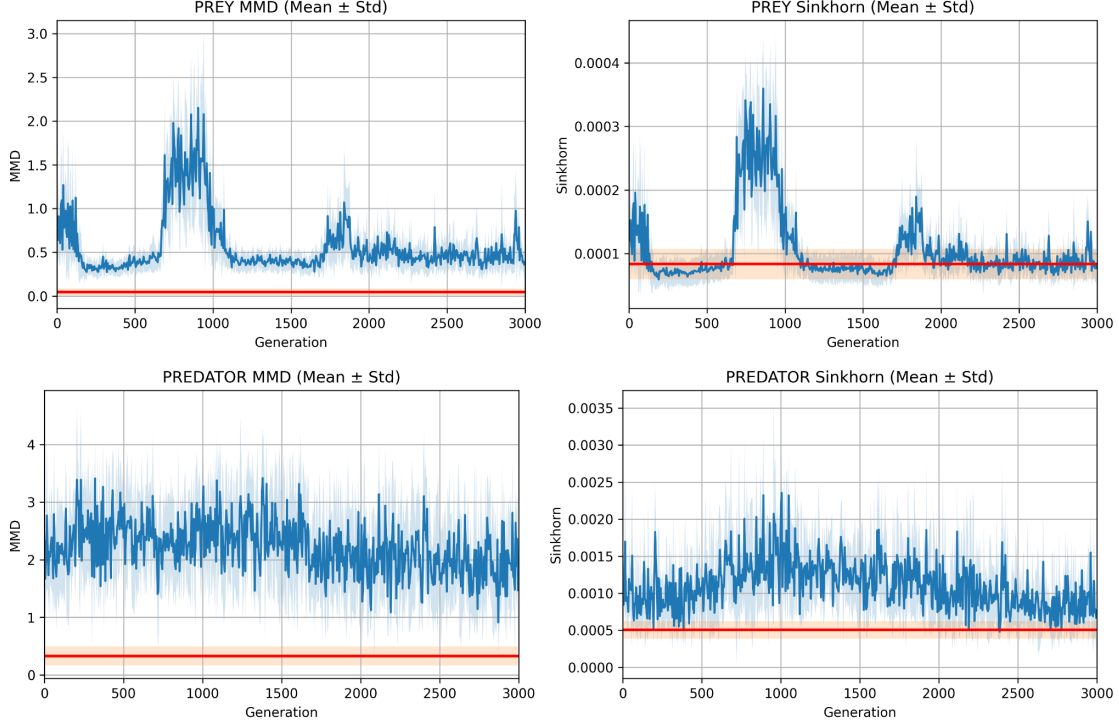


Figure 5.16: MMD and Sinkhorn distances over generations for the video predator-prey model, focusing only on attack data.

Restricting training to only successful attacks produces the best distribution match obtained in this work. The prey policy reaches its expert baseline under the Sinkhorn distance and holds it, which neither of the previous stages managed to do, and the predator improves steadily without attaining its own baseline.

5.5. Fine-Tuning with a Joint Discriminator

All previous stages trained a separate discriminator for each role, so the critic does not observe the relationship between the predator and the prey. To address this, a joint discriminator (Section 4.6.1) was added as a fine-tuning stage on top of the converged attack-only models. Starting from an already converged model preserves the stability achieved in earlier stages, while allowing the reward signal to depend on the joint state of both roles. The attack-only models were chosen as the starting point because they had the strongest results (Section 5.4), although performance was better for the

prey than for the predator. Given this asymmetry, the joint discriminator was then used with the specific aim of improving the predator’s behavior.

A sweep of 12 configurations was run to test whether any hyperparameter choices improve the joint discriminator. All configurations were initialized from the same checkpoint and trained for 300 generations, so that differences in performance can be attributed to the hyperparameters alone.

The sweep covered four groups of design choices:

1. Discriminator strength relative to the policies

- Number of discriminator updates per policy update (Arjovsky et al., 2017)
- Update frequency of the discriminator
- Warm-up period for the discriminator at the start of training
- Periodic cooldown phases during which discriminator updates were paused

2. Discriminator regularization

- Weight of the gradient penalty, tested in both directions (Gulrajani et al., 2017)
- Magnitude of instance noise added to the discriminator inputs (Sønderby et al., 2017)

3. Policy optimization

- Separate learning rates for predator and prey, following a two-timescale update rule (Heusel et al., 2017)
- Magnitude of the exploration noise from the evolution strategy
- Decay applied to both the learning rates and the exploration noise over training (Salimans et al., 2017)

4. Training schedule

- Freezing the prey policy so that only the predator adapts (Bengio et al., 2009)
- Early stopping based on the evaluation score

Across the sweep, results fell within a narrow band, and the ranking of configurations differed between the two roles. Compared to the separately trained models, the joint discriminator improves the MMD for both roles but worsens the Sinkhorn distance, most clearly for the predator. Escape alignment moves closer to the expert, but not by more than the variation observed between repeated evaluations. Overall, the joint discriminator does not close the gap in inter-group behavior, and the analyses that follow are therefore based on the Stage 1 attack-only policies.

Evaluation of Learned Predator-Prey Behavior

Beyond the metrics used to evaluate training, additional analyses are conducted to examine how closely the learned policies reproduce real predator-prey behavior. This chapter focuses on the policy trained on the attack-only subset of the video data, which includes both the 16 and 32 prey conditions. First, pursuit and evasion are analyzed, as these are the behaviors that characterize predation (Peterson et al., 2021). Different metrics are used for the predator and prey, since each role exhibits differing behavior during an attack. Second, the collective motion of the swarm is evaluated using swarm-level metrics. This quantifies how aligned the group is internally and provides insights to the interactions within the group during an attack. Third, the modular policy networks are examined to better understand the learned policies, using heatmaps to illuminate the internal workings of these otherwise black-box networks.

6.1. Pursuit and Evasion

In every attack video clip, the predator pursues the prey and executes a successful strike, while the prey tries to evade capture. Because the policy was trained on exactly this type of data, effective learning should reproduce these pursuit and evasion patterns and closely match the expert behavior. Predator pursuit is characterized by three measures: nearest-prey distance, pursuit alignment, and closing speed. For the prey, the following metrics are used: distance to predator, escape alignment, and prey reorientation. Together these metrics capture the core dynamics of pursuit and evasion. For every metric, the GAIL model is compared against the expert data and two baselines: the behavioral cloning model and the Couzin simulation model. All of the models are evaluated in a single 200-step simulation and since rollouts must be simulated at a single group size, the behavioral evaluation is carried out with 16 prey. Metrics that evolve over time are plotted against the expert

mean and a band of ± 1 standard deviation across all attack videos and metrics that are summarized as distributions are shown against an expert boxplot.

6.1.1. Predator Pursuit

In expert fish behavior, successful capture is when predators pursue prey and then strike. There are several ways to quantify this predation.

6.1.1.1. Nearest-Prey Distance

The first measure considered is the nearest-prey distance, so the distance between the predator and its closest prey over a period of time. The proximity of the predator and the prey is the most important factor for the prey surviving (Peterson et al., 2021). Prey farthest away from the predator when starting to flee have a high chance of survival, whereas those who let the predator close the distance before reacting usually were the target of an attack. This is supported by the expert videos, where the predator tended to capture a prey that was closer to it. Since the policy is learned on successful attacks, this number should be relatively low throughout.

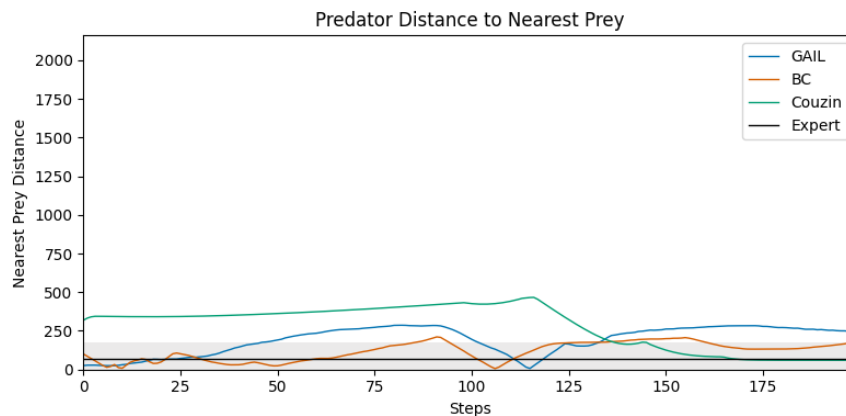


Figure 6.1: Predator distance to nearest prey model comparison.

In the expert data, this distance between the predator and its nearest prey stays small throughout the labeled clips, reflecting that these clips capture the final approach of an attack (Figure 6.1). The behavior cloning model stays closest to the expert range, moving between around 0 and 220 px. The GAIL model stays in the expert range for around the first 25 steps, then drifts up to 250

px, with another approach to the expert around step 115. The Couzin model begins the furthest away and increases until around step 125, after which it closes rapidly and stays near the prey for the remainder of the simulation.

6.1.1.2. Pursuit Alignment

The second measure is pursuit alignment, which is how well the predator's heading aligns with the direction to its nearest prey. The heading refers to the direction that the agent is facing and moving towards, which corresponds to its focus direction (Soto & McHenry, 2020; Peterson et al., 2021). The policy would be expected to show the predator being oriented towards the position of the prey.

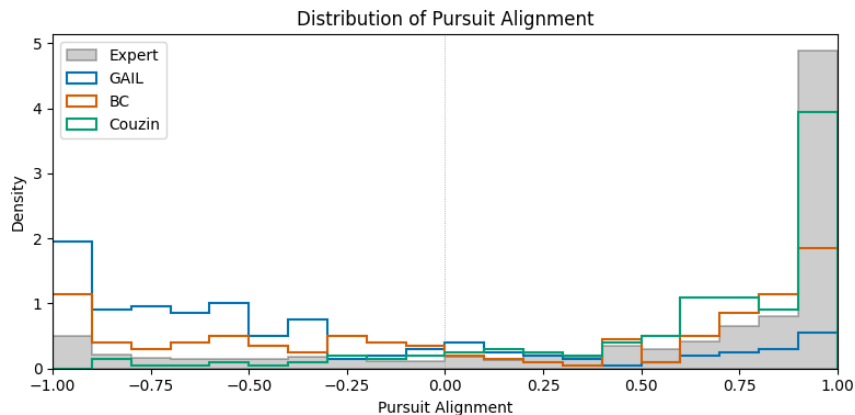


Figure 6.2: Distribution of pursuit alignment model comparison.

Figure 6.2 shows that the expert distribution of pursuit alignment is strongly concentrated near +1, meaning that the predator is oriented almost directly at its closest prey in the majority of labeled frames. The Couzin model shows a similar pattern, which is expected because it steers deterministically towards the closest prey. Both the behavioral cloning model and the GAIL model instead show comparable densities at +1 and -1, with GAIL placing its highest density at -1, meaning the learned predator orients away from the nearest prey more than towards it.

6.1.1.3. Closing Speed

The third measure is closing speed, which measures how quickly the predator approaches its nearest prey, directly shaping the prey's escape response. The predator's approach speed significantly affects

the distance at which prey initiate their escape (Stewart et al., 2014). When predators close the gap quickly, prey have less time to assess the threat and react before being caught. As a result, higher closing speeds are associated with shorter response distances and reduced escape success. All agents move at a fixed speed, so closing speed does not mean the predator is speeding up or slowing down. It only reflects how the predator and prey are positioned and moving: when the predator is aimed at prey swimming straight away, the distance does not shrink, but when the prey have not yet reacted, the distance shrinks quickly. The policy would therefore be expected to show positive closing speeds whenever it is oriented towards prey that are not yet escaping effectively.

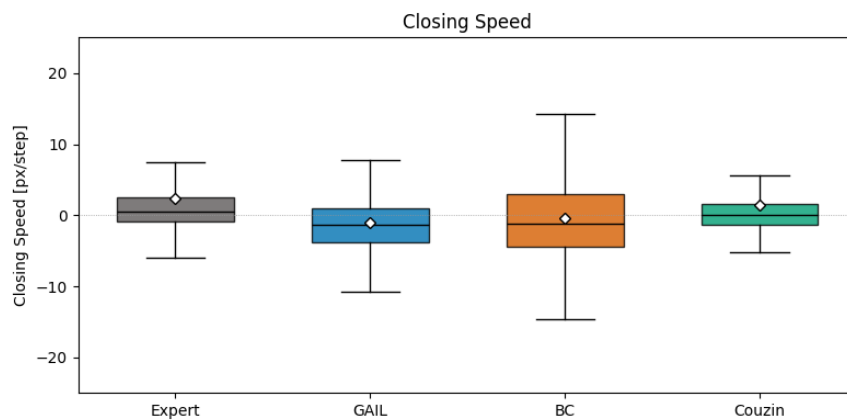


Figure 6.3: Closing speed model comparison.

Seen in Figure 6.3, the expert closing speed has a mean slightly above zero, meaning a slow net approach to prey across frames. The Couzin model is the closest match to the expert with a similar center and spread. The behavioral cloning model’s mean is centered near zero but with a wider spread and the GAIL model’s mean is shifted slightly below zero. All three models sit close to the expert, and they differ from each other more in spread than in center.

6.1.2. Prey Evasion

From the prey’s perspective, successful evasion is when individuals detect an approaching predator and avoid capture. There are several ways to quantify this evasion.

6.1.2.1. Distance to Predator

The first measure is distance to predator, which is the distance between the predator and the center of mass of the school. Since it is computed from the center of the group rather than from an individual prey, it shows avoidance at the level of the school as a whole. Being near a predator is an immediate threat for the prey, so the prey try to maximize their distance away from the predator (Bartashevich et al., 2024).

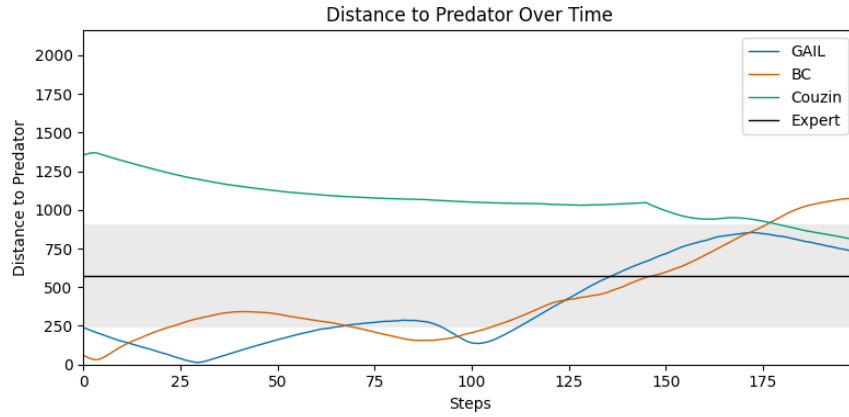


Figure 6.4: Distance to predator model comparison.

In Figure 6.4, the expert’s average distance to predator is around 600 px in the labeled frames. The behavior cloning model and the GAIL model follow similar patterns across the simulation, starting off with the predator close to the prey school and then increasing steadily from around step 100. This measure cannot distinguish whether the school moves away from the predator or the predator moves away from the school.

6.1.2.2. Escape Alignment

The second measure is escape alignment, which quantifies how strongly the prey are heading away from the predator. It is computed as the mean over all prey of the cosine between a prey’s heading and the direction pointing from the predator to that prey. A value of +1 means every prey is swimming directly away, 0 means the headings do not show a preferred direction relative to the predator, and -1 means the prey are heading towards the predator, so effective escape corresponds

to a positive value. Escape direction is a central component of a prey’s escape strategy (Soto et al., 2015). In what is called a fountain effect, the school splits around an approaching predator and reforms behind it, so the escape direction of an individual depends on the direction of the attack of the predator (Bartashevich et al., 2024). This means that prey do not flee along one axis and the expert values should be only moderately positive instead of closer to +1.



Figure 6.5: Escape alignment model comparison.

In Figure 6.5, the expert data mean escape alignment is around 0.27, well below the maximum of +1, confirming that the prey do not all flee along a single axis away from the predator. The behavior cloning model follows the expert most closely. After the first ten steps, it stays within one standard deviation of the expert mean for almost the entire simulation. The GAIL model tracks the expert band for the first 60 steps, then falls to approximately -0.45 between steps 75 and 95, meaning the prey are oriented towards the predator rather than away, before recovering to the expert range at around step 110 onwards. The Couzin model remains below the expert mean throughout the simulation and turns negative from around step 85, only returning to positive values towards the end.

6.1.2.3. Prey Reorientation

The third measure is prey reorientation, which is how strongly the prey turn towards or away from the predator between consecutive frames. Positive values mean the prey turned further away from the predator over the step, while negative values mean they turned towards it. With fish, an escape

response starts with a quick turn away from the threat (Domenici & Blake, 1997) and the direction of the escape maneuver determines the escape success (Domenici, 2002). Since the simulated agents move at a constant speed, the focus here is on the direction component of the escape response rather than acceleration.

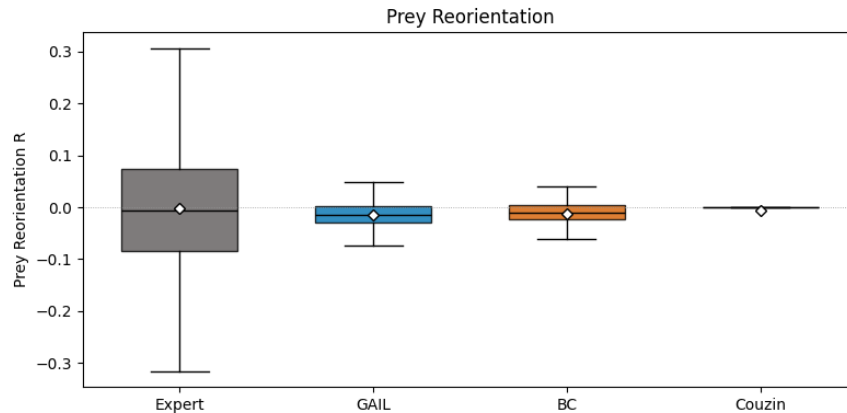


Figure 6.6: Prey reorientation model comparison.

In Figure 6.6, the distributions from the expert as well as all three of the models are centered on zero, which means that over a simulation, turns towards and away from a predator are canceled out. The expert has the widest distribution, meaning that the prey change orientation relative to the predator by larger amounts between consecutive frames. The behavior cloning and GAIL models produce almost identical and narrow distributions, while the Couzin model is almost a flat line, indicating that essentially its prey does not reorient relative to the predator at all.

All in all, these six metrics show that the GAIL model reproduces expert pursuit and evasion behavior over a limited time horizon. It remains close to the expert during the first part of the simulation, which corresponds to the length of the labeled clips, and diverges afterwards. Its performance is closely matched by the behavioral cloning model across all six measures, so adversarial training provides no clear advantage over supervised imitation based on these metrics.

6.2. Collective Motion within Swarm

Beyond individual pursuit and evasion, imitation quality is also assessed at the swarm level. This matters for predator-prey interactions because individual actions can scale up into coordinated, group-wide behavior. Múgica et al. (2022) showed that collective turns started by only a small subset of fish can spread through the entire school as behavioral cascades, turning a local reaction into a large group-wide response. Risk itself is also partly a collective property. Sosna et al. (2019) found that risk is encoded not only in how strongly individuals respond, but also in the spatial structure of the group, which shapes how information about a threat such as a predator is transmitted between individuals. Individual responses and group structure are therefore tightly coupled, with each fish's actions depending on its position in the school and these reactions in turn shaping the structure of the whole group. Since the policy is trained on predator-prey interactions that contain this group-level structure, a learned policy should be able to reproduce the collective motion observed in the expert data. However, it is important to note that the expert data used for training consists exclusively of attack sequences. During attacks, behavior can be more erratic and chaotic in response to the predator than during undisturbed schooling. It is still important to analyze these collective motions to see how the group behaves before and after the attack. These measures are therefore expected to show more moderate values than in undisturbed schools. Three collective motion measures are used to capture this structure: polarization, degree of sparsity, and angular momentum.

6.2.1. Polarization

The first measure is polarization, which quantifies how aligned the school is as a whole (Li et al., 2023; Yapei Wu et al., 2024). Specifically, it measures how strongly the fish align their headings with one another. Headings in the expert data are approximated by each fish's direction of motion. A value of 1 means that all fish swim in exactly the same direction, while lower values that are closer to 0 indicate more disordered motion. Polarity is one of the defining characteristics of a fish school that strongly influences the members within the swarm, yet it is a property that no fish controls directly (Viscido et al., 2004). They further observed that real fish schools were strongly

aligned most of the time, interrupted by some irregular periods where the school broke down into a disorganized swarm, and that polarized groups tend to move faster than non-polarized ones (ibid.). At the moment of an attack, polarization is impacted because the swarm splits around the predator when the predator strikes, causing the polarization to temporarily drop (Bartashevich et al., 2024). Later, the group needs time to reorganize, regroup, and return to a cohesive polarized group after an attack.

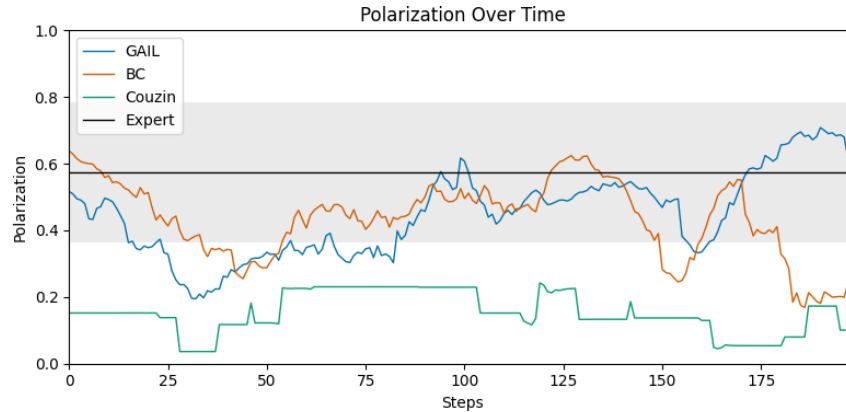


Figure 6.7: Polarization model comparison.

In the expert data, the mean sits around 0.55 (Figure 6.7), meaning that the school is aligned, though not perfectly. This is consistent with the clips being focused on the exact attack moment, as they do not show undisturbed schooling. The GAIL model remains within one standard deviation of the expert mean for most of the simulation. The behavioral cloning model tracks the expert mean until step 160, where it falls to 0.2 and does not recover. The Couzin model performs the worst, remaining far below the expert throughout the simulation. It does not produce an aligned school.

6.2.2. Degree of Sparsity

The second measure is degree of sparsity, which quantifies how spread out the swarm is (Li et al., 2023). It is calculated as the mean distance between each individual and its nearest neighbor, and can be interpreted as a distance measure in pixels. Lower values mean a compact and dense swarm with closely packed neighbors, whereas higher values indicate larger gaps between individuals and their nearest neighbors, so a more loosely spaced and separated swarm. In addition to polarity,

the degree of sparsity is another main defining characteristic of fish schools that strongly affects their members (Viscido et al., 2004), and it may shape how a school of prey collectively responds to predators. During a predator-prey interaction, when a predatory threat is initially detected, the prey typically decrease their distance to their neighbors and the school becomes a more compact and tight swarm. This behavior is thought to reduce each individual’s chances of getting captured by the predator (Magurran & Pitcher, 1987). During an attack, this structure can break down. Since attacks are associated with increased fragmentation in the spatial structure of swarms, the degree of sparsity may also vary with prey dispersing away from the predator (Handegard et al., 2012). This fragmentation also inhibits the transfer of information between prey (ibid.).

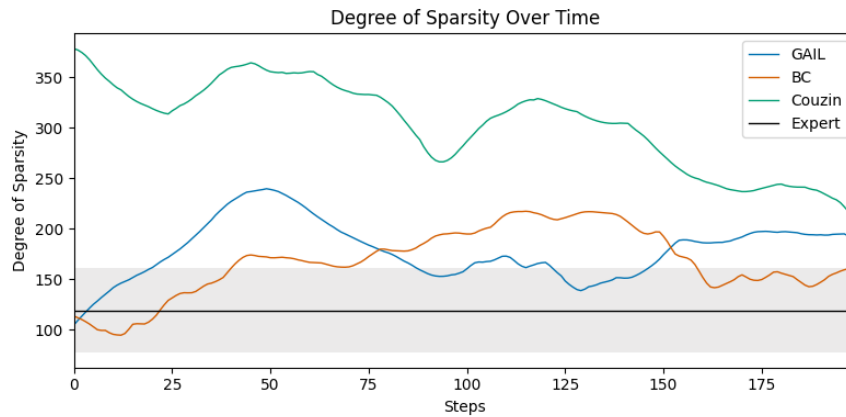


Figure 6.8: Degree of sparsity model comparison.

As seen in Figure 6.8, the expert mean is around 120 px. This is consistent with a compact school during the attack sequences that make up the expert data. Both the GAIL model and the behavioral cloning model begin at expert levels, then expand, both ending up fluctuating just above the upper edge of the expert band. The Couzin model is between 250 and 375 px throughout, roughly two to three times the expert value, meaning the swarms are more spread out.

6.2.3. Angular Momentum

The third measure is angular momentum, which quantifies how strongly the school rotates around its own center, which is a pattern known as milling (Couzin et al., 2002; Yapei Wu et al., 2024). For each prey, the cross product between its position relative to the group’s center of mass and its

heading direction is computed, and values are expressed in pixels. The absolute value of the mean over all prey results in the group’s angular momentum, with the predator excluded. Higher values mean that milling behavior occurs and the fish circle the center in the same rotational direction. Lower values mean that there is no collective rotation around the center, and the fish either move in parallel or in random directions. Milling is one of several main collective behaviors seen in fish swarms, along with disordered swarms and polarized motions (Tunstrøm et al., 2013).

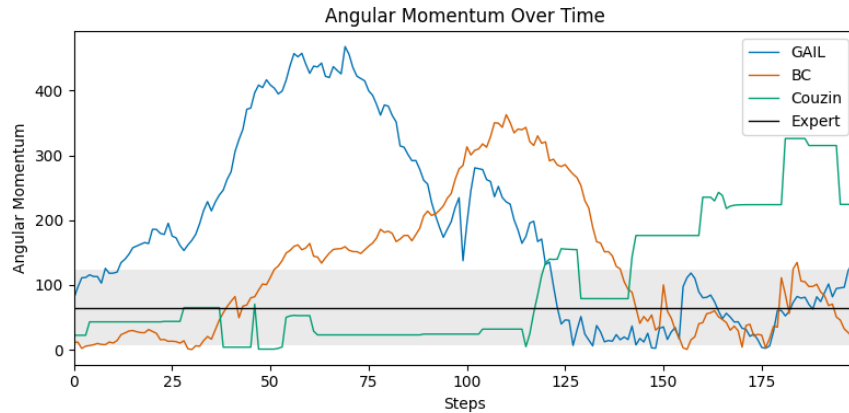


Figure 6.9: Angular momentum model comparison.

The expert mean is around 62 px, low as expected, given that the expert data consists only of successful attacks (Figure 6.9). Both the GAIL model and the behavioral cloning model show an extended milling episode far above the expert range, but in both cases they return back to expert levels and remain there for the rest of the simulation. Therefore milling is seen in the learned models as a transient phase instead of a sustained state. The Couzin model shows the opposite pattern. It stays in the expert range until step 115 then rises around step 125, staying elevated until the end of the simulation.

These three collective metrics show that GAIL model reproduces the structure of the expert school considerably more closely than the Couzin model, staying near the expert polarization and only slightly above the expert sparsity for most of the simulation. The behavioral cloning model performs comparably, although it is the only model whose polarization collapses before the end of

the simulation, while GAIL remains within the expert range throughout. Milling appears in both learned models as a temporary phase rather than a sustained state.

6.3. Analysis of Policy Networks

To better understand the learned policy, the two components of the modular policy, namely the pairwise interaction network (PIN) and the attention network (AN), were examined in more detail. Both networks were introduced in Section 4.1. Separate response maps were created for the PIN and AN so that each decision process could be analysed on its own. Because the policy's actions are based on a weighted combination of contributions from each neighbor, each sub-network can be queried individually. The PIN shows how a single neighbor in a given position would influence the focal individual's movement. The AN shows how much weight that neighbor would receive compared to the others. This decomposition enables interaction rules to be read directly from the trained model, which would not be possible with a single network. This approach follows that of Yapei Wu et al. (2024), who analyzed policy networks trained using Couzin simulation data. Here, following Wirtheim (2026), the same analysis is applied to networks trained on real expert attack data rather than simulated data.

Maps are created for both the PIN and the AN. Each map shows, on average, how the focal fish would respond if a neighbor were at a given position. Each map is generated by evaluating the network on a 100×100 grid of relative neighbor positions spanning $[-1, 1]$ in both directions, with positions normalized by the arena size. At every grid point, the neighbor's velocity is assigned 100 evenly spaced headings, and the outputs are averaged over these headings so that each map depends on the neighbor's position alone. The neighbor's speed is fixed at 0.15, the median neighbor speed observed in the expert recordings, rather than at the maximum of the normalization range, so that the maps describe the policy's response in the regime it was trained on. For the PIN, the color shows the turn the environment would apply in response to that single neighbor. Positive values (red) indicate a counter-clockwise turn in the coordinate frame of the map and negative values (blue) a clockwise turn. For the AN, the colors show the relative importance of the neighbor, min-max normalized within each map from 0 (least important) to 1 (most important). Three maps

are produced: the predator's response to prey, the prey's response to other prey, and the prey's response to the predator.

6.3.1. Predator Reacting to Prey PIN and AN Maps

In the following maps, the predator is the focal individual and each point represents a possible position of a single prey fish. Looking at the PIN map, prey positioned to the left and upper left drive the predator counter-clockwise, while prey in the lower-right quadrant drive it clockwise. Over large parts of the map, the response lies near the maximum turn the environment permits, so a single prey pushes the predator to one of the two extremes. In the AN map, attention increases towards the left and downwards. Prey along the left and lower-left of the map therefore receive the most weight, and a distant prey on that side is weighted more heavily than a closer prey on the opposite side.

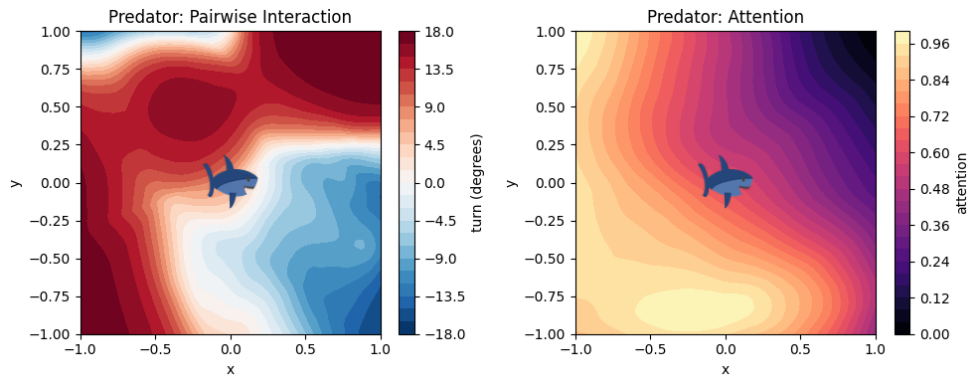


Figure 6.10: Predator response to prey PIN and AN maps.

6.3.2. Prey Reacting to Prey PIN and AN Maps

In the following maps, a prey is the focal individual and each point represents a possible position of one other prey fish. In the PIN map, the response is counter-clockwise across the left half and the upper right, and more clockwise in the lower-right corner. The region immediately around the focal individual stays closer to zero. The strongest turns are produced by distant conspecifics, while nearby ones lead only to small course corrections. The AN map on the other hand has its highest values centered on the focal individual and decreases with distance. Weighting among conspecifics

is therefore distance-based. Together, the two maps show that most of the weight falls on neighbors that produce the smallest turns.

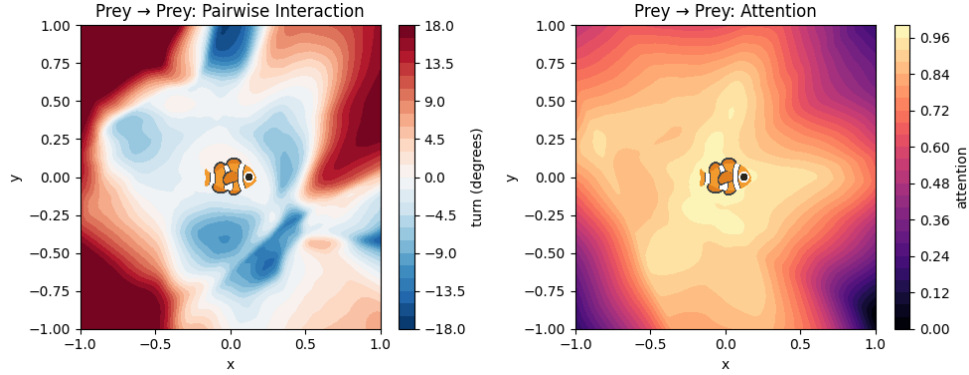


Figure 6.11: Prey response to other prey PIN and AN maps.

6.3.3. Prey Reacting to Predator PIN and AN Maps

In the following maps, the prey is the focal individual and each point on the map represents a possible position of the predator. The PIN map looks clearly different from the one where the prey reacts to another prey. The response is clockwise across the left half and the lower right, and strongly counter-clockwise in the upper-right corner. As in the previous map, the response is weakest close to the focal individual and strongest towards the edges. The AN map has its highest values at the focal individual, but extends it further to the left, and low weights are confined to the very right of the map. Because the only difference between these maps and those in Section 6.3.2 is the neighbor-type flag, the contrast between them shows that the policy has learned a predator-specific rule rather than treating the predator as just another neighbor.

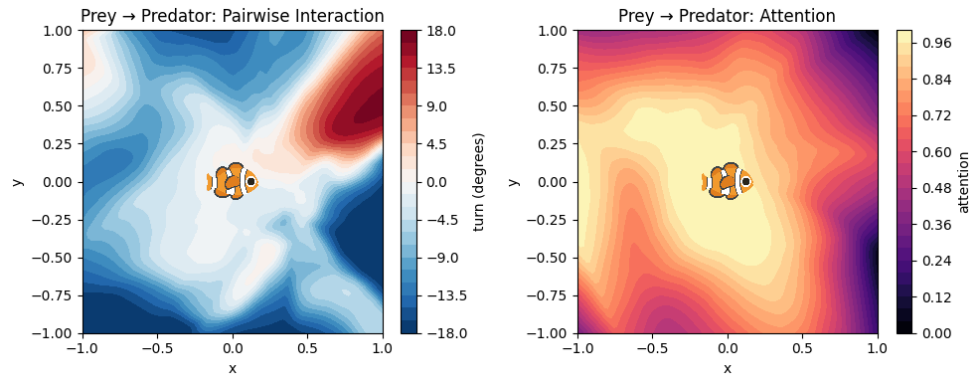


Figure 6.12: Prey response to the predator PIN and AN maps.

Discussion

Wirtheim (2026) reported partial convergence and no evidence of pursuit or evasion in the predator-prey setting. This thesis investigates whether these interactions are recoverable at all, and whether the limiting factor lies in the data, the policy architecture, or the training objective.

Training proceeded in stages of increasing complexity, starting with Couzin-generated predator-prey data, then real video predator-prey data, and finally video predator-prey data restricted to successful attacks (Chapter 5). GAIL training on Couzin predator-prey data verified that the framework could recover predator-prey behavior from clean synthetic demonstrations. It trained well with the 16- and 32-prey conditions combined, showing that the group-size-agnostic tensor representation remains stable under adversarial training. Moving onto using real video data, both distribution metrics stabilize and Sinkhorn approaches the expert baseline for the predator and the prey, in contrast to the partial convergence and the absence of evasion or pursuit reported by Wirtheim (2026) for the predator-prey setting.

While several changes separate the two settings, the most substantial is the training data: the 16-prey dataset constructed for this thesis raises the available windows from 793 to 3,771 and the data consists only of attack and meaningful interactions instead of also including undisturbed schooling. The rollouts showed the predator successfully pursuing a prey, but the prey group did not maintain tight cohesion. To test if this remaining gap came from the composition of the data, the following step was to restrict training to successful attacks. Behavioral cloning on the attack-only data already yielded more consistent tracking than all previous stages, with the predator following the prey throughout the rollout. This is the first indication that training data composition, not architecture, governs whether directed interaction is recovered. Looking at the GAIL results, the attack-only prey policy reached its expert baseline under the Sinkhorn distance and held it, while

the predator approached its own baseline albeit with some noise. This supports the hypothesis motivating this stage by showing that the constraint in the previous stages can be attributed to the low proportion of attack events in the training distribution rather than to the model’s capacity. Notably, this improvement was achieved with less data: 2,200 attack windows in the current stage versus 3,771 in the previous stage.

Focusing the training distribution on the behavior of interest was therefore more effective than increasing the dataset size. Despite this progress, the predator trajectories still exhibited noticeable noise, indicating that the remaining difficulty is concentrated in the predator role. To address this, a joint discriminator was introduced with the specific aim of improving the predator’s behavior (Section 5.5). However, fine-tuning with the joint discriminator did not close the gap in inter-group behavior, so all subsequent analyses are based on the Stage 1 attack-only policies.

In addition to the training evaluation metrics, further analysis is needed to assess whether proper imitation occurred. A set of metrics was used to analyze predator pursuit, prey evasion, and swarm behavior (Sections 6.1.1 and 6.1.2). Predator pursuit is quantified by three measures: nearest-prey distance, pursuit alignment, and closing speed. For nearest-prey distance and closing speed, both the GAIL and behavioral cloning (BC) models remain close to the expert range. In pursuit alignment, both learned models show the predator oriented almost directly toward the nearest prey about half the time and facing away the other half, whereas the expert predator is oriented toward the prey in the majority of labeled frames. This discrepancy may reflect the difference in time horizon: simulations run for 200 steps, while the labeled clips are only 63 frames long and end shortly after a strike, so most simulated steps fall outside the situations represented in the training data. Prey evasion is quantified by distance to predator, escape alignment, and prey reorientation. In distance to predator, both GAIL and BC follow similar patterns, with the predator starting close to the school and then steadily moving away. Read together with pursuit alignment, where the GAIL predator drifts outward after its approach, this increase is more plausibly driven by the predator than by a collective escape of the prey. For escape alignment, both GAIL and BC remain close to the expert range. Swarm behavior is characterized by polarization, degree of sparsity, and angular momentum.

For polarization and degree of sparsity, both GAIL and BC are close to expert levels. Milling appears in both learned models as a temporary phase rather than a sustained state, consistent with expert data drawn from short attack sequences rather than undisturbed schooling.

Overall, these metrics provide evidence that the models learned key aspects of pursuit and evasion, and some deviations can be attributed to the longer simulation horizon. Notably, GAIL and BC perform very similarly across all metrics, except for polarization: GAIL stays near the expert band, while BC collapses to about 0.2 around step 160. This pattern suggests that, for pursuit and evasion, adversarial training offers no clear advantage over behavioral cloning. Moving on, policy maps were used to better understand the learned policy. The policy maps show that predator-specific structure is built into the network itself, not just in the overall trajectories: the same prey network produces a clearly different response and weighting once a neighbor is flagged as the predator. At the same time, the two roles weigh their neighbors differently, with prey prioritizing distance and predators prioritizing direction, consistent with schooling behaviors.

These findings, however, come with limitations that outline clear next steps for future work. First, both predator and prey move at a constant speed in the simulations. In reality, pike, which is the predator species used in this study, captures prey through rapid accelerations during strikes (Harper & Blake, 1991), a dynamic that the current constant-speed model cannot reproduce. Second, additional research could go into the specific attack style of pike and what aspects of that behavior are suitable for imitation learning. Pike exhibit S-start type attacks (Schriefer & Hale, 2004), which may require different metrics or representations to capture well. Third, Li’s research group did collective trials on sunbleak groups of different sizes (1, 2, 4, 8, 16, 32, or 64 individuals). Future work could maximize labeling of attack data across all group-size conditions, which would substantially increase the amount of training data. Now that a group-size-agnostic tensor representation and a model that trains adversarially on this data are in place, the pipeline can readily accommodate additional labeled data. Fourth, the attack clips were limited to 5 seconds. Given that focusing training on the target attack behavior yields the best results, extending the length of attack videos and prioritizing attack footage over other interaction clips may be more beneficial. Fifth, it could

be worth exploring the joint discriminator architecture more. For instance, this study used max pooling, but alternatives such as mean pooling or attention mechanisms could be attempted.

Beyond ecological modeling, the methods developed here can serve as a foundation for other applications. For instance, the trained policies could be transferred to robots to learn decentralized control policies for robot swarms (Yapei Wu et al., 2024) or adapted to real-world scenarios such as crowd management and evacuation planning.

Conclusion

This thesis set out to determine whether predator-prey interactions can be recovered from real video recordings using imitation learning, and specifically whether pursuit and escape can be modeled. Wirthheim (2026) did not capture these inter-group dynamics, so this work investigated whether they are recoverable at all, and whether the limiting factor lies in the data, the policy architecture, or the training objective. Building on his framework, this thesis constructed a hand-labeled 16-prey dataset with a group-size-agnostic representation, limited training to successful attacks to avoid diluting the target behavior, and explored a new policy network and a joint discriminator for fine-tuning the existing architecture.

The methodology begins with a data-processing pipeline that converts raw recordings into relative state representations of the interactions, through feature engineering and tensor construction. This tensor representation is deliberately agnostic to group size, so the same format can be reused regardless of how many prey appear in a given scene. These expert tensors are then used to train a policy through GAIL, in which predator and prey each receive their own policy, transition encoder, and discriminator. The GAIL and evolution-strategies implementation is adapted into a framework in which the predator and prey policies are optimized as two separate, modular subproblems, so that each role can be trained and evaluated on its own terms.

To evaluate the models, training evaluation metrics were combined with additional metrics for pursuit and evasion, for swarm-level behavior, and with policy maps for the modular networks. Training succeeded on the video predator-prey data, and convergence was even stronger on data focused specifically on attacks. These results indicate that the density of the target behavior in the training data matters more than its volume. Restricting training to successful attacks produced the

best distribution match of any configuration tested, and the pursuit-and-evasion metrics confirm that the learned models reproduce this behavior over the time span covered by the labeled clips.

Future work can look into a model that reproduces varying speeds, since the current model assumes a constant speed; better metrics for evaluating pike-specific attacks; and more attack data, in order to further improve training on the target behavior. Overall, this work shows that predator-prey interactions can be recovered from video, provided training is concentrated on the densest examples of the target behavior rather than spread across all available data.

APPENDIX

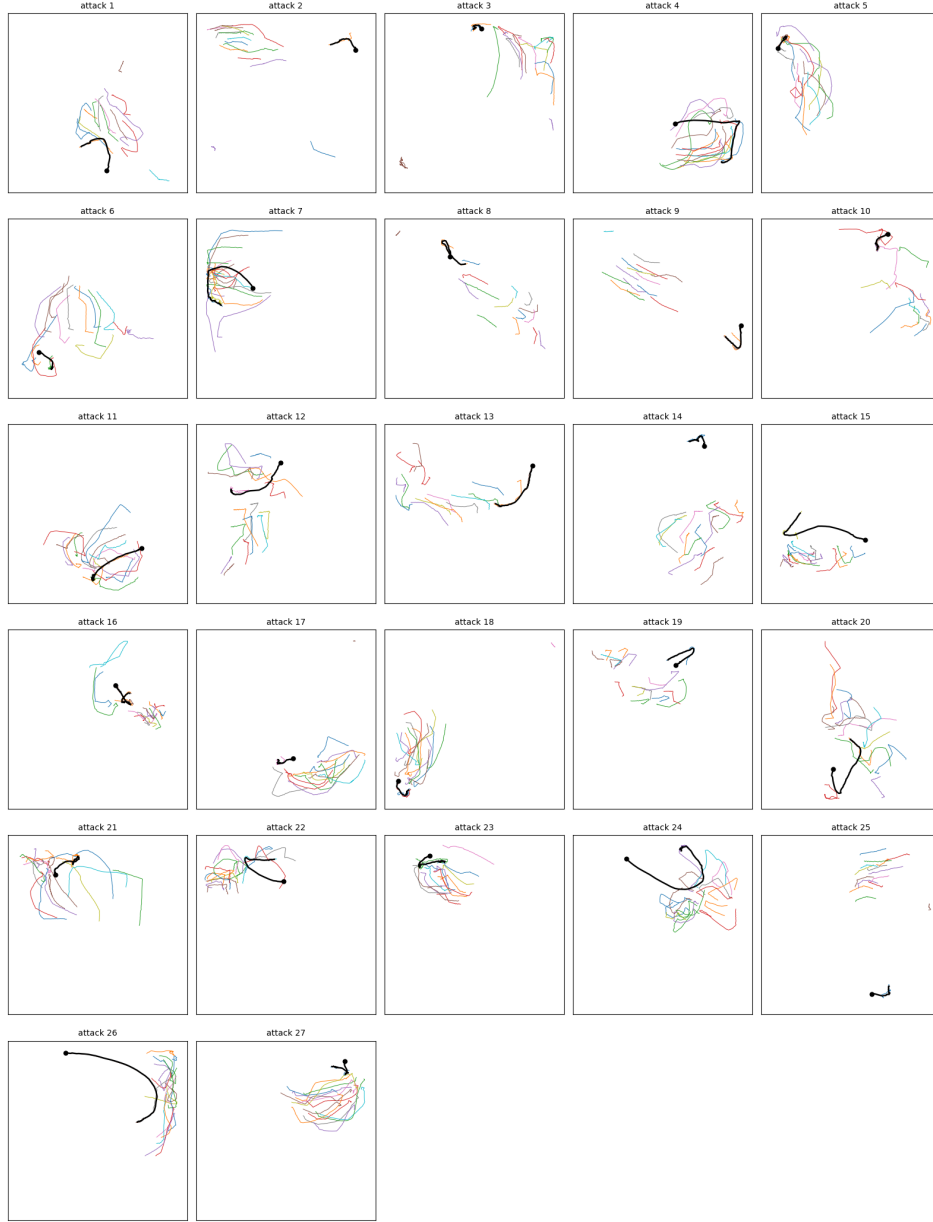


Figure A.1: Reconstructed trajectories for all 27 hand-labeled 16-prey attack clips.

Figure A.2: Overview of hyperparameters used for the Couzin predator-prey model.

Parameter	Value
Expert Data Generation	
Max steps per episode (<code>max_steps</code>)	300
Training Setup	
Number of generations (<code>num_generations</code>)	3000
Decay factor (<code>gamma</code>)	0.999
Pretraining (<code>pretrain</code>)	False
Performance evaluation interval (<code>performance_eval</code>)	5
Prey Policy	
Prey policy learning rate (<code>lr_prey_policy</code>)	2×10^{-4}
Number of perturbations (<code>num_perturbations</code>)	64
Prey exploration noise (<code>sigma_prey</code>)	0.1
Prey Discriminator	
Prey discriminator-policy ratio (<code>prey_dis_balance_factor</code>)	2
Prey input noise (<code>prey_noise</code>)	0.005
Prey discriminator learning rate (<code>lr_prey_disc</code>)	5×10^{-4}
Prey gradient penalty (<code>lambda_gp_prey</code>)	5
Prey update mode (<code>prey_update_mode</code>)	mean ($\lambda = 0.2$)
Predator Policy	
Predator policy learning rate (<code>lr_pred_policy</code>)	1×10^{-4}
Predator exploration noise (<code>sigma_pred</code>)	0.08
Predator Discriminator	
Predator discriminator-policy ratio (<code>pred_dis_balance_factor</code>)	2
Predator input noise (<code>pred_noise</code>)	0.005
Predator discriminator learning rate (<code>lr_pred_disc</code>)	2×10^{-4}
Predator gradient penalty (<code>lambda_gp_pred</code>)	10
Predator update mode (<code>pred_update_mode</code>)	mean ($\lambda = 0.1$)
Environment Settings	
Arena height (<code>height</code>)	50
Arena width (<code>width</code>)	50
Prey speed (<code>prey_speed</code>)	5
Predator speed (<code>pred_speed</code>)	5
Step size (<code>step_size</code>)	0.5
Max turn per step (<code>max_turn</code>)	0.25
Rollout Configuration	
Perturbation steps (<code>pert_steps</code>)	100
Initial steps (<code>init_steps</code>)	500

Figure A.3: Overview of hyperparameters used for the video predator-prey model and the attack-only video predator-prey model.

Parameter	Value
Training Setup	
Number of generations (<code>num_generations</code>)	3000
Decay factor (<code>gamma</code>)	0.999
Pretraining (<code>pretrain</code>)	True
Performance evaluation interval (<code>performance_eval</code>)	5
Prey Policy	
Prey policy learning rate (<code>lr_prey_policy</code>)	2×10^{-4}
Number of perturbations (<code>num_perturbations</code>)	64
Prey exploration noise (<code>sigma_prey</code>)	0.1
Prey Discriminator	
Prey discriminator-policy ratio (<code>prey_dis_balance_factor</code>)	2
Prey input noise (<code>prey_noise</code>)	0.005
Prey discriminator learning rate (<code>lr_prey_disc</code>)	5×10^{-4}
Prey gradient penalty (<code>lambda_gp_prey</code>)	5
Prey update mode (<code>prey_update_mode</code>)	mean ($\lambda = 0.2$)
Predator Policy	
Predator policy learning rate (<code>lr_pred_policy</code>)	1×10^{-4}
Predator exploration noise (<code>sigma_pred</code>)	0.08
Predator Discriminator	
Predator discriminator-policy ratio (<code>pred_dis_balance_factor</code>)	2
Predator input noise (<code>pred_noise</code>)	0.005
Predator discriminator learning rate (<code>lr_pred_disc</code>)	1×10^{-4}
Predator gradient penalty (<code>lambda_gp_pred</code>)	10
Predator update mode (<code>pred_update_mode</code>)	mean ($\lambda = 0.1$)
Environment Settings	
Arena height (<code>height</code>)	2160
Arena width (<code>width</code>)	2160
Prey speed (<code>prey_speed</code>)	10
Predator speed (<code>pred_speed</code>)	10
Step size (<code>step_size</code>)	1.0
Max turn per step (<code>max_turn</code>)	0.314
Velocity normalization bound (<code>max_speed_norm</code>)	25
Rollout Configuration	
Perturbation steps (<code>pert_steps</code>)	100
Initial steps (<code>init_steps</code>)	500

Figure A.4: Python libraries used in this thesis.

Package	Version	Reference
deep-sort-realtime	1.3.2	(Wojke, 2023)
ema-pytorch	0.7.9	(Wang, 2025)
geomloss	0.2.6	(Feydy et al., 2019)
matplotlib	3.10.3	(Hunter, 2007)
numpy	2.0.2	(Harris et al., 2020)
opencv-python	4.12.0.88	(Bradski, 2000)
pandas	2.3.1	(Team, 2020)
scipy	1.15.3	(Virtanen et al., 2020)
torch	2.6.0+cu124	(Paszke et al., 2019)
tqdm	4.67.1	(da Costa-Luis, 2019)
ultralytics	8.3.164	(Jocher and Contributors, 2025)

Bibliography

- Arjovsky, M., Chintala, S., & Bottou, L. (2017). Wasserstein gan. <https://doi.org/10.48550/arXiv.1701.07875>
- Arora, S., & Doshi, P. (2020). A survey of inverse reinforcement learning: Challenges, methods and progress. *arXiv preprint arXiv:1811.06584*.
- Bardes, A., Ponce, J., & LeCun, Y. (2021). VICReg: Variance-invariance-covariance regularization for self-supervised learning. *arXiv preprint arXiv:2105.04906*.
- Bartashevich, P., Herbert-Read, J. E., Hansen, M. J., Dhellemmes, F., Domenici, P., Krause, J., & Romanczuk, P. (2024). Collective anti-predator escape manoeuvres through optimal attack and avoidance strategies. *Communications Biology*, 7(1), 1586. <https://doi.org/10.1038/s42003-024-07267-2>
- Bengio, Y., Louradour, J., Collobert, R., & Weston, J. (2009). Curriculum learning. *Proceedings of the 26th Annual International Conference on Machine Learning*, 41–48.
- Bradski, G. (2000). The OpenCV library.
- Calovi, D. S., Lopez, U., Schuhmacher, P., Chaté, H., Sire, C., & Theraulaz, G. (2015). Collective response to perturbations in a data-driven fish school model. *Journal of The Royal Society Interface*, 12(104), 20141362.
- Couzin, I. D., & Krause, J. (2003). Self-organization and collective behavior in vertebrates. *Advances in the Study of Behavior*, 32, 1–75.
- Couzin, I. D., Krause, J., James, R., Ruxton, G. D., & Franks, N. R. (2002). Collective memory and spatial sorting in animal groups. *Journal of Theoretical Biology*, 218(1), 1–11.
- da Costa-Luis, C. (2019). *Tqdm* (Version 4.67.1).
- Di Marzo Serugendo, G., Foukia, N., Hassas, S., Karageorgos, A., Mostéfaoui, S. K., Rana, O. F., Ulieru, M., Valckenaers, P., & Van Aart, C. (2003). Self-organisation: Paradigms and applications. *International Workshop on Engineering Self-Organising Applications*, 1–19.
- Domenici, P., & Blake, R. W. (1997). The kinematics and performance of fish fast-start swimming. *Journal of Experimental Biology*, 200(8), 1165–1178.

- European Commission. (2026a). Commercial designations and scientific name of *Esox lucius*. Retrieved August 31, 2026, from https://fish-commercial-names.ec.europa.eu/fish-names/species/esox-lucius_de
- European Commission. (2026b). Commercial designations and species information for *Leucaspis delineatus*. Retrieved August 31, 2026, from https://fish-commercial-names.ec.europa.eu/fish-names/species/leucaspis-delineatus_en
- Feydy, J., S  journ  , T., Vialard, F.-X., Amari, S.-i., Trouv  , A., & Peyr  , G. (2019). Interpolating between optimal transport and MMD using Sinkhorn divergences. *Proceedings of the 22nd International Conference on Artificial Intelligence and Statistics (AISTATS)*.
- Goodfellow, I. J., Pouget-Abadie, J., Mirza, M., Xu, B., Warde-Farley, D., Ozair, S., Courville, A., & Bengio, Y. (2014). Generative adversarial networks. *Advances in Neural Information Processing Systems*, 2672–2680.
- Gui, J., Chen, T., Zhang, J., Cao, Q., Sun, Z., Luo, H., & Tao, D. (2024). A survey on self-supervised learning: Algorithms, applications, and future trends. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 46(12), 9052–9071.
- Gulrajani, I., Ahmed, F., Arjovsky, M., Dumoulin, V., & Courville, A. (2017). Improved training of wasserstein GANs.
- Haeri, H. (2026). Couzin swarm model: Predator-prey simulation. https://github.com/hossein-haeri/couzin_swarm_model/blob/master/swarm_pray_predator.py
- Handegard, N. O., Boswell, K. M., Ioannou, C. C., Leblanc, S. P., Tjostheim, D. B., & Couzin, I. D. (2012). The dynamics of coordinated group hunting and collective information transfer among schooling prey. *Current Biology*, 22(13), 1213–1217. <https://doi.org/10.1016/j.cub.2012.04.050>
- Harper, D. G., & Blake, R. W. (1991). Fast-start performance of rainbow trout *Salmo gairdneri* and northern pike *Esox lucius*. *Journal of Experimental Biology*, 150(1), 321–342.
- Harris, C. R., Millman, K. J., van der Walt, S. J., Gommers, R., Virtanen, P., Cournapeau, D., Wieser, E., Taylor, J., Berg, S., Smith, N. J., et al. (2020). Array programming with NumPy. *Nature*, 585(7825), 357–362.

- Heras, F. J. H., Romero-Ferrero, F., Hinz, R. C., & de Polavieja, G. G. (2019). Deep attention networks reveal the rules of collective motion in zebrafish. *PLOS Computational Biology*, *15*(9), 1–23.
- Heusel, M., Ramsauer, H., Unterthiner, T., Nessler, B., & Hochreiter, S. (2017). Gans trained by a two time-scale update rule converge to a local nash equilibrium. <https://arxiv.org/abs/1706.08500>
- Ho, J., & Ermon, S. (2016). Generative adversarial imitation learning. *Advances in Neural Information Processing Systems*, *29*.
- Hunter, J. D. (2007). Matplotlib: A 2d graphics environment. *Computing in Science & Engineering*, *9*(3), 90–95.
- Hussein, A., Gaber, M. M., Elyan, E., & Jayne, C. (2017). Imitation learning: A survey of learning methods. *ACM Computing Surveys*, *50*(2), 1–35.
- Jocher, G., & Contributors. (2025). *Ultralytics yolov8* (Version 8.3.164).
- Kuhn, H. W. (1955). The hungarian method for the assignment problem. *Naval Research Logistics Quarterly*, *2*(1–2), 83–97.
- Leu, T., Lüscher, B., Zumbach, S., & Schmidt, B. R. (2009). Small fish (*Leucaspius delineatus*) that are often released into garden ponds and amphibian breeding sites prey on eggs and tadpoles of the common frog (*Rana temporaria*). *Amphibia-Reptilia*, *30*(2), 290–293.
- Li, J., Li, L., & Zhao, S. (2023). Predator–prey survival pressure is sufficient to evolve swarming behaviors. *New Journal of Physics*, *25*(9), 092001.
- Li, L. (2025). Behavioral fish trajectory dataset [Data access provided by the Department of Collective Behavior at the Max Planck Institute of Animal Behavior].
- Liu, G., Zhao, L., Yang, F., Bian, J., Qin, T., Yu, N., & Liu, T.-Y. (2019). Trust region evolution strategies. *Proceedings of the AAAI Conference on Artificial Intelligence*, *33*, 4352–4359.
- Luise, G., Rudi, A., Pontil, M., & Ciliberto, C. (2018). Differential properties of sinkhorn approximation for learning with wasserstein distance.

- Magurran, A. E., & Pitcher, T. J. (1987). Provenance, shoal size and the sociobiology of predator-evasion behaviour in minnow shoals. *Proceedings of the Royal Society of London B*, 229, 439–465.
- Marras, S., & Domenici, P. (2013). Schooling fish under attack are not all equal: Some lead, others follow. *PLoS ONE*, 8(6), e65784.
- Mehraban, Z., & Pichler, A. (2025). Quantization of probability measures in maximum mean discrepancy distance.
- Morales-Brotons, D., Vogels, T., & Hendriks, H. (2024). Exponential moving average of weights in deep learning: Dynamics and benefits.
- Múgica, J., Torrents, J., Cristín, J., Puy, A., Miguel, M. C., & Pastor-Satorras, R. (2022). Scale-free behavioral cascades and effective leadership in schooling fish. *Scientific Reports*, 12(1), 10783. <https://doi.org/10.1038/s41598-022-14337-0>
- Osa, T., Pajarinen, J., Neumann, G., Bagnell, J. A., Abbeel, P., & Peters, J. (2018). An algorithmic perspective on imitation learning. *Foundations and Trends in Robotics*, 7(1-2), 1–179.
- Paszke, A., Gross, S., Massa, F., Lerer, A., Bradbury, J., Chanan, G., Killeen, T., Lin, Z., Gimelshein, N., Antiga, L., et al. (2019). PyTorch: An imperative style, high-performance deep learning library. *Advances in Neural Information Processing Systems* 32, 8024–8035.
- Peterson, A. N., Soto, A. P., & McHenry, M. J. (2021). Pursuit and evasion strategies in the predator–prey interactions of fishes. *Integrative and Comparative Biology*, 61(2), 668–680.
- Pierson, E., & Kao, P. (2025). Markov decision processes [In preparation]. *arXiv preprint*.
- Reynolds, C. W. (1987). Flocks, herds and schools: A distributed behavioral model. *Proceedings of the 14th Annual Conference on Computer Graphics and Interactive Techniques (SIGGRAPH '87)*, 25–34.
- Ross, S., Gordon, G. J., & Bagnell, J. A. (2011). A reduction of imitation learning and structured prediction to no-regret online learning. *AISTATS*, 627–635.
- Salimans, T., Ho, J., Chen, X., Sidor, S., & Sutskever, I. (2017). Evolution strategies as a scalable alternative to reinforcement learning.

- Schafer, T. L. J., Wikle, C. K., & Hooten, M. B. (2022). Bayesian inverse reinforcement learning for collective animal movement. *arXiv preprint*.
- Schriefer, J. E., & Hale, M. E. (2004). Strikes and startles of northern pike (*Esox lucius*): A comparison of muscle activity and kinematics between s-start behaviors. *Journal of Experimental Biology*, 207(3), 535–544.
- Sønderby, C. K., Caballero, J., Theis, L., Shi, W., & Huszár, F. (2017). Amortised map inference for image super-resolution. <https://arxiv.org/abs/1610.04490>
- Song, J., Ren, H., Sadigh, D., & Ermon, S. (2018). Multi-agent generative adversarial imitation learning. *Advances in Neural Information Processing Systems*, 31.
- Sosna, M. M. G., Twomey, C. R., Bak-Coleman, J., Poel, W., Daniels, B. C., Romanczuk, P., & Couzin, I. D. (2019). Individual and collective encoding of risk in animal groups. *Proceedings of the National Academy of Sciences*, 116(41), 20556–20561. <https://doi.org/10.1073/pnas.1905585116>
- Soto, A. P., Stewart, W. J., & McHenry, M. J. (2015). When optimal strategy matters to prey fish. *Integrative and Comparative Biology*, 55(1), 110–120. <https://doi.org/10.1093/icb/icv027>
- Soto, A. P., & McHenry, M. J. (2020). Pursuit predation with intermittent locomotion in zebrafish. *Journal of Experimental Biology*, 223(11), jeb230623.
- Stewart, W. J., Nair, A., Jiang, H., & McHenry, M. J. (2014). Prey fish escape by sensing the bow wave of a predator. *Journal of Experimental Biology*, 217(24), 4328–4336. <https://doi.org/10.1242/jeb.108068>
- Sumpter, D. J. T. (2006). The principles of collective animal behaviour. *Philosophical Transactions of the Royal Society B: Biological Sciences*, 361(1465), 5–22.
- Team, T.P.D. (2020). *Pandas-dev/pandas: Pandas* (Version 2.3.1).
- Tieleman, T., & Hinton, G. (2012). Lecture 6.5—RMSProp: Divide the gradient by a running average of its recent magnitude [Lecture slides; often listed as Lecture 6e/6.5 in the course materials].
- Tkachenko, M., Malyuk, M., Holmanyuk, A., & Liubimov, N. (2020–2025). Label studio: Data labeling software [Open-source software]. <https://github.com/HumanSignal/label-studio>

- Torabi, F., Warnell, G., & Stone, P. (2018). Behavioral cloning from observation. *arXiv preprint arXiv:1805.01954*.
- Tunström, K., Katz, Y., Ioannou, C. C., Huepe, C., Lutz, M. J., & Couzin, I. D. (2013). Collective states, multistability and transitional behavior in schooling fish. *PLoS Computational Biology*, 9(2), e1002915. <https://doi.org/10.1371/journal.pcbi.1002915>
- Ultralytics. (2026a). Explore ultralytics YOLOv8. <https://docs.ultralytics.com/models/yolov8>
- Ultralytics. (2026b). Ultralytics YOLO docs. <https://docs.ultralytics.com>
- Veličković, P., Cucurull, G., Casanova, A., Romero, A., Lio, P., & Bengio, Y. (2017). Graph attention networks. *arXiv preprint arXiv:1710.10903*.
- Vicsek, T., Czirók, A., Ben-Jacob, E., Cohen, I., & Shochet, O. (1995). Novel type of phase transition in a system of self-driven particles. *Physical Review Letters*, 75(6), 1226–1229.
- Virtanen, P., Gommers, R., Oliphant, T. E., Haberland, M., Reddy, T., Cournapeau, D., Burovski, E., Peterson, P., Weckesser, W., Bright, J., et al. (2020). SciPy 1.0: Fundamental algorithms for scientific computing in Python. *Nature Methods*, 17(3), 261–272.
- Viscido, S. V., Parrish, J. K., & Grünbaum, D. (2004). Individual behavior and emergent properties of fish schools: A comparison of observation and theory. *Marine Ecology Progress Series*, 273, 239–249. <https://doi.org/10.3354/meps273239>
- Wang, Y. (2025). *Ema-pytorch* (Version 0.7.9).
- Wang, Y., Hayashibe, M., & Owaki, D. (2024). Data-driven policy learning methods from biological behavior: A systematic review. *Applied Sciences*, 14(10), 4038. <https://doi.org/10.3390/app14104038>
- Wirthheim, J. (2026). *Exploring predator–prey dynamics from videos using generative adversarial imitation learning* [Master’s thesis]. University of Konstanz.
- Wojke, N. (2023). *Deep-sort-realtime* (Version 1.3.2). <https://pypi.org/project/deep-sort-realtime/>
- Wu, Y., Wang, T., Liu, T., Zheng, Z., Xu, D., & Peng, X. (2024). Adversarial imitation learning with deep attention network for swarm systems. *Complex & Intelligent Systems*, 11(1), 26.

- Wu, Y., Dou, Z., Ishiwaka, Y., Ogawa, S., Lou, Y., Wang, W., Liu, L., & Komura, T. (2025). CBIL: Collective behavior imitation learning for fish from real videos. *ACM Transactions on Graphics*, 43(6), 1–17.
- Yong, C. H., & Miikkulainen, R. (2001). Cooperative coevolution of multi-agent systems. *Proceedings of the Genetic and Evolutionary Computation Conference*, 1–8.
- Yu, X., Wu, W., Feng, P., & Tian, Y. (2021). Swarm inverse reinforcement learning for biological systems. *2021 IEEE International Conference on Bioinformatics and Biomedicine (BIBM)*, 274–279.
- Zhang, M., Wang, Y., Ma, X., Xia, L., Yang, J., Li, Z., & Li, X. (2020). Wasserstein distance guided adversarial imitation learning with reward shape exploration. *2020 IEEE 9th Data Driven Control and Learning Systems Conference (DDCLS)*, 1165–1170.